



Top-down mining of frequent closed patterns from very high dimensional data [☆]

Hongyan Liu ^{a,*}, Xiaoyu Wang ^a, Jun He ^b, Jiawei Han ^c, Dong Xin ^c, Zheng Shao ^c

^a Department of Management Science and Engineering, Tsinghua University, Beijing 100084, China

^b Department of Computer Science, Renmin University of China, Beijing 100872, China

^c Department of Computer Science, University of Illinois at Urbana-Champaign, Urbana, IL 61801, USA

ARTICLE INFO

Article history:

Received 15 February 2008

Received in revised form 25 November 2008

Accepted 26 November 2008

Keywords:

Data mining

Association rules

Frequent patterns

High dimensional data

ABSTRACT

Frequent pattern mining is an essential theme in data mining. Existing algorithms usually use a bottom-up search strategy. However, for very high dimensional data, this strategy cannot fully utilize the minimum support constraint to prune the rowset search space. In this paper, we propose a new method called top-down mining together with a novel row enumeration tree to make full use of the pruning power of the minimum support constraint. Furthermore, to efficiently check if a rowset is closed, we develop a method called the *trace-based* method. Based on these methods, an algorithm called *TD-Close* is designed for mining a complete set of frequent closed patterns. To enhance its performance further, we improve it by using new pruning strategies and new data structures that lead to a new algorithm *TTD-Close*. Our performance study shows that the top-down strategy is effective in cutting down search space and saving memory space, while the trace-based method facilitates the closeness-checking. As a result, the algorithm *TTD-Close* outperforms the bottom-up search algorithms such as *Carpenter* and *FPClose* in most cases. It also runs faster than *TD-Close*.

© 2008 Elsevier Inc. All rights reserved.

1. Introduction

Frequent pattern mining is an essential theme in data mining, with broad applications [9,13,14]. There are many frequent pattern mining algorithms that have been proposed, but most of them [1,10–12,21,22,26,27] deal with transactional databases that usually contain a huge number of rows (or tuples) but only a small number of dimensions (or attributes, columns). These algorithms usually search the *itemset* (i.e., set of items) space; therefore, they are called **column enumeration-based algorithms**. However, many applications may involve another kind of database, which is characterized by a small number of rows but a large number of dimensions. We call this kind of data *very high dimensional data*. Gene expression matrices analysis in bioinformatics [2,3,5,15,18,25] and text processing [23] are examples of this kind of application.

Due to the exponential number of combinations of dimensions, very high dimensional data poses great challenges on efficient frequent pattern mining as to most other data mining algorithms. To solve the problem, a new approach is proposed to

[☆] A preliminary version of this work has been published in the Proceedings of 2006 SIAM Data Mining Conference (SIDM 2006). Substantial new technical materials have been added to this journal submission. This work was supported in part by the National Natural Science Foundation of China under Grant Nos. 70871068, 70890083 and 70621061, and by the US National Science Foundation NSF IIS-03-08215 and IIS-05-13857.

* Corresponding author. Tel.: +86 10 6278 9961; fax: +86 10 6278 4555.

E-mail addresses: hylu@tsinghua.edu.cn (H. Liu), wangxy3@sem.tsinghua.edu.cn (X. Wang), hejun@ruc.edu.cn (J. He), hanj@uiuc.edu (J. Han), dongxin@uiuc.edu (D. Xin).

search the **rowset** (i.e., set of rows) space rather than the itemset space for frequent pattern mining [19,24]. We call these kinds of algorithms **row enumeration-based algorithms**. Based on this work, several other algorithms can also be developed for frequent pattern mining or interesting rule group mining [3,4,20].

The two kinds of algorithms mentioned above adopt a bottom-up search strategy that searches from small itemsets to large itemsets. For column enumeration-based algorithms, bottom-up search can use the anti-monotone property of minimum support threshold [1] to prune the search space, which means that if an itemset of size k is not frequent, any of its super sets of size $(k + 1)$ is not frequent either. However, for row enumeration-based algorithms, bottom-up search may suffer from the following two inefficiencies:

- (1) The potential search space is large. This is due to the fact that bottom-up search in the rowset space cannot utilize the anti-monotone property of minimum support threshold to prune the space. As a result, many unnecessary searches need to be done.
- (2) The data set needs to be checked repeatedly to see if an itemset is closed.

In this paper, we propose a top-down search strategy for row enumeration-based algorithms to overcome these drawbacks. To do that, we develop and integrate the following four techniques:

- (1) A novel row enumeration tree is constructed. By traversing this tree, a top-down search of the rowset space is conducted, that is, the rowsets are examined from the large ones to the small ones. This style of search order can take advantage of the anti-monotone heuristic to prune the search space. For example, rowsets smaller than k can be pruned if the minimum support threshold is set at k .
- (2) A divide-and-conquer top-down search technique is proposed to mine frequent closed itemsets efficiently. Using this technique, the search space can be partitioned into separate subspaces and handled recursively by the top-down search technique.
- (3) A new *trace-based closeness-checking* method is developed to check whether an itemset is closed. Further, a *output-based method* is designed based on a traditional closeness-checking method and is further improved according to the characteristics of the new top-down search order. Our experiments show that the trace-based *closeness-checking* method performs more efficiently than the output-based method. Unlike other existing closeness-checking methods, the trace-based *closeness-checking* method does not need to scan the data set or the result set of itemsets repeatedly. Moreover, this method can be easily integrated with the top-down search process.
- (4) Several other pruning strategies are also developed. These strategies can reduce the search space further.

Integrating the above four techniques, an algorithm called *TD-Close* was designed and implemented by an array-based data structure. To improve its performance further, another algorithm called *TTD-Close* was then developed. It uses more effective pruning strategies aside from those used in *TD-Close*, and it is implemented with a tree-based data structure. A comprehensive experimental study was conducted on both synthetic data sets and real world data sets to compare the performance of these two algorithms as well as with two other frequent pattern mining algorithms: (1) *FPclose* [7], a representative column enumeration-based algorithm, and (2) *Carpenter* [19], a representative row enumeration-based algorithm. Our experimental results confirm the effectiveness of the new trace-based *closeness-checking method* and the improvement of *TTD-Close* over *TD-Close*. Furthermore, both analysis and experimental results indicate that using the top-down search method, *TD-Close* and *TTD-Close* can cut down more search space and use less memory space than *Carpenter* especially when the minimum support is high. It can also achieve better performance than *FPclose* when the minimum support is low.

The remainder of the paper is organized as follows: Section 2 introduces the transposition method. Section 3 describes the proposed top-down search technique. Section 4 presents the new *closeness-checking* approach. Section 5 develops the two new frequent closed pattern mining algorithms based on the top-down search technique and trace-based *closeness-checking* approach. Section 6 presents our performance study, and Section 7 describes the related work. Finally, Section 8 concludes our study and discusses some future research issues.

2. Frequent pattern mining by transposition

2.1. Definition

Let T be a discretized data table (or data set) composed of a set of n rows $\mathcal{S} = \{r_1, r_2, \dots, r_n\}$, where r_j ($j = 1, \dots, n$) is a row ID or *rid* for short. Each row corresponds to a sample consisting of a set of discrete values or intervals, and $\mathcal{V}$ is the complete set of these values or intervals $\mathcal{V} = \{i_1, i_2, \dots, i_m\}$. For simplicity, we call each i_k ($k = 1, \dots, m$) an *item* and a set of items $I \subseteq \mathcal{V}$ an **itemset**. An itemset with k items is called a ***k*-itemset**.

Example 1 (Table T). Table 1 shows an example of table T with four dimensions (columns): A, B, C, and D. For simplicity, we use number j ($j = 1, 2, \dots, n$) instead of r_j to represent each *rid*. This table contains five rows and nine items, so $\mathcal{S} = \{1, 2, 3, 4, 5\}$ and $\mathcal{V} = \{a_1, a_2, b_1, b_2, c_1, c_2, d_1, d_2, d_3\}$.

Table 1An example of table T .

Row ID	A	B	C	D
1	a_1	b_1	c_1	d_1
2	a_1	b_1	c_2	d_2
3	a_1	b_1	c_1	d_2
4	a_2	b_1	c_2	d_2
5	a_2	b_2	c_2	d_3

Definition 1 (*Transposed Table*). Given a table $T(\mathcal{S}, \mathcal{V})$, where $\mathcal{S} = \{r_1, r_2, \dots, r_n\}$, $\mathcal{V} = \{i_1, i_2, \dots, i_m\}$, the *transposed table* T^t of T is composed of m tuples, each of which is identified by an item $i_k \in \mathcal{V}$ and consists of a set of *rids*. If tuple i_k contains r_j in T^t , it means item i_k is contained by row r_j in Table T .

Therefore, the transposed table of a table with m distinct items contains m rows. For clarity, we call each of these rows a **tuple**. Each tuple corresponds to an item i_k from $\mathcal{V}$ and consists of a set of *rids* that contain i_k in T . We can regard each item i_k as the tuple ID of each tuple in T^t .

Similar to the definitions in table T , we call a set of *rids* $X \subseteq \mathcal{S}$ a **rowset** and a rowset having k *rids* a **k -rowset** in the transposed table T^t .

Definition 2 (*Row ID Order*). In the transposed table T^t , the row IDs in each tuple are ordered by a predefined order. We call this order *row ID order* or *rid order*.

For example, we can define *rid order* according to the size of each *rid* in T . We say that a row r_j is greater than r_k if row ID r_j is ordered after row ID r_k according to specific predefined row ID order, and a row r_j is less than r_k if row ID r_j is ordered before row ID r_k .

Example 2 (*Transposed table and rid order*). Table 2 shows the transposed table of table T shown in Table 1. In this transposed table, we define the numerical order of *rids* as the *rid order*. We say that *rid* 2 is greater than *rid* 1 and less than *rid* 3.

Suppose the minimum support threshold (denoted by *minsup*; its definition will be given in Definition 5) is set at 2, i.e., all tuples with the number of *rids* less than 2 is deleted. Note that the table T^t shown in Table 2 is already pruned by the *minsup*. This kind of pruning will be further explained in the following sections.

Definition 3 (*Closure*). For a table $T(\mathcal{S}, \mathcal{V})$, given an itemset $I \subseteq \mathcal{V}$ and a rowset $X \subseteq \mathcal{S}$, we define

$$r(I) = \{r_j \in \mathcal{S} \mid \forall i_k \in I, r_j \text{ contains } i_k \text{ in } T\}, \quad (1)$$

$$i(X) = \{i_k \in \mathcal{V} \mid \forall r_j \in X, r_j \text{ contains } i_k \text{ in } T\}. \quad (2)$$

Based on this definition, we define $C(I)$ as the closure of an itemset I and $C(X)$ as the closure of a rowset X as follows:

$$C(I) = i(r(I)), \quad (3)$$

$$C(X) = r(i(X)). \quad (4)$$

Note that Definition 3 is also applicable to the transposed table T^t .

Definition 4 (*Closed itemset and closed rowset*). An itemset I is a **closed itemset** if $I = C(I)$. Likewise, a rowset X is a **closed rowset** if $X = C(X)$.

Definition 5 (*Frequent itemset and large rowset*). Given an absolute value of the user-specified threshold *minsup*, an itemset I is **frequent** if $|r(I)| \geq \text{minsup}$ and a rowset X is **large** if $|X| \geq \text{minsup}$, where $|r(I)|$ is the *support* of itemset I , *minsup* is the minimum support threshold, and $|X|$ is the size of rowset X accordingly.

Furthermore, an itemset I is a *frequent closed itemset* if it is both closed and frequent. Likewise, a rowset X is a *large closed rowset* if it is both closed and large.

Table 2Transposed table T^t of T .

Tuple ID	Rowset
a_1	1, 2, 3
a_2	4, 5
b_1	1, 2, 3, 4
c_1	1, 3
c_2	2, 4, 5
d_2	2, 3, 4

Example 3 (Closed itemset and closed rowset). In Table 1, for an itemset $\{b_1, c_2\}$, $r(\{b_1, c_2\}) = \{2, 4\}$ and $i(\{2, 4\}) = \{b_1, c_2, d_2\}$, so $C(\{b_1, c_2\}) = \{b_1, c_2, d_2\}$. Therefore, $\{b_1, c_2\}$ is not a closed itemset. If the $minsup = 2$, it is a frequent itemset. In Table 2, for a rowset $\{1, 2\}$, $i(\{1, 2\}) = \{a_1, b_1\}$ and $r(\{a_1, b_1\}) = \{1, 2, 3\}$, then $C(\{1, 2\}) = \{1, 2, 3\}$. Therefore, rowset $\{1, 2\}$ is not a closed rowset but apparently $\{1, 2, 3\}$ is.

2.2. Frequent pattern mining

In our dataset, the number of rows is much smaller than the number of the dimensions. Thus, the combination of rows is much less than the combination of dimensions. Therefore, it is likely that transposition for the original data set may reduce search complexity. However, here is a key question: “Can we find the same complete set of frequent itemsets in the transposed table T^t as from the original table T given the minimum support threshold $minsup$?”

To answer this question, we can start by analyzing the original mining task and traditional mining method. Originally, we want to find all the frequent itemsets that satisfy the minimum support threshold $minsup$ from table T .

Example 4 (Frequent itemsets). For table T shown in Table 1, the complete set of frequent itemsets supposing the $minsup$ is set at 2 is listed in Table 3. These itemsets can be mined directly by any existing frequent pattern mining algorithm such as Apriori [1] and FP-growth [10].

It can be seen from Table 3 that every frequent itemset has one corresponding rowset consisting of a set of rows in which the itemset occurs. Moreover, if an itemset has support that is not less than the $minsup$, then the size of its corresponding rowset is also equal to or greater than the $minsup$. Therefore, can we get a frequent itemset and its support value from each rowset with a size not less than the $minsup$?

The answer is negative. For example, suppose the $minsup = 2$, in transposed table T^t shown in Table 2, we can see that rowset $\{1, 2\}$ has a size 2 that satisfies the $minsup$ and has itemset $\{a_1, b_1\}$ related to it. Therefore, itemset a_1b_1 is frequent. Unfortunately, its support is not equal to 2 as it appears not only in row 1 and 2 but also in row 3. Hence, inferring frequent itemset directly from a large rowset would get the wrong support value.

Moreover, sometimes one rowset corresponds to more than one itemset. For instance, from Table 2 we can see that rowset $\{1, 3\}$ relates to one itemset $\{a_1, b_1, c_1\}$, but Table 3 tells us that it relates to three itemsets, $\{c_1\}$, $\{b_1, c_1\}$ and $\{a_1, b_1, c_1\}$. It is easy to find the large rowset $\{1, 3\}$ and its related itemset $\{a_1, b_1, c_1\}$, but it is not easy to find the other two itemsets directly from this rowset.

Further study of the relationship between itemsets and rowsets indicates that each closed itemset relates to one closed rowset, and each closed rowset is related to one closed itemset as shown in Lemma 1.

Lemma 1. Each closed itemset I_s has a related closed rowset X , which contains each of the items in itemset I_s . Similarly, each closed rowset X has a related closed itemset I_s , which occurs in each row of rowset X .

Proof. Since itemset I_s is closed according to Definition 4, we have $I_s = i(r(I_s))$. Let $X = r(I_s)$, which stands for a set of rows containing each item of I_s . Now we can prove that rowset X must be closed. If X is not closed, then there must exist another rowset $S \supset X$ which appears in the same set of tuples in table T^t , where each tuple corresponds to one item of itemset I_s . This means $r(I_s) = S$, which contradicts to $X = r(I_s)$. Likewise, we can prove that for each closed rowset X , there exists a closed itemset $I_s = i(X)$, which occurs in each row of rowset X . From Lemma 1, we know that the closed itemsets of table T and closed rowsets of table T^t have a one-to-one relationship as illustrated in Example 5. $\square$

Example 5 (one-to-one relationship). In Table 3, the itemsets and rowsets shown in bold with grey background are closed. It is easy to see that each closed itemset maps to a unique closed rowset and vice versa.

Based on Lemma 1, we have the following Lemma 2:

Lemma 2 Given table T and its transposed table T^t for a user given threshold $minsup$, we can find all the frequent closed itemsets of table T by searching for all the large closed rowsets from T^t .

Table 3
Frequent itemsets for table T when $minsup = 2$.

1-Itemset	Rowset	2-Itemset	Rowset	3-Itemset	Rowset
a_1	1, 2, 3	a_1b_1	1, 2, 3	$a_1b_1c_1$	1, 3
a_2	4, 5	a_1c_1	1, 3	$a_1b_1d_2$	2, 3
b_1	1, 2, 3, 4	a_2c_2	4, 5	$b_1c_2d_2$	2, 4
c_1	1, 3	a_1d_2	2, 3		
c_2	2, 4, 5	b_1c_1	1, 3		
d_2	2, 3, 4	b_1c_2	2, 4		
		b_1d_2	2, 3, 4		
		c_2d_2	2, 4		

Proof. According to Definition 3 and Lemma 1, since each closed rowset corresponds to a closed itemset, if we can find all of the closed rowsets with a size no less than the *minsup* from the transposed table T^t of the original table T , then we can get all the closed itemsets directly.

Now we can answer the question raised at the beginning of this section. Frequent closed itemsets are a lossless compression of frequent itemsets [21], i.e., a complete set of frequent itemsets can be derived from a complete set of frequent closed itemsets. Therefore, we can first find all the large closed rowsets from the transposed table T^t and get the frequent closed itemsets in the meantime, and then derive the frequent itemsets from the frequent closed itemsets. $\square$

3. Search strategies

To mine a complete set of large closed rowsets from the transposed table T^t , we need to develop an effective search strategy for efficient mining. The existing frequent pattern mining algorithms, either column enumeration-based [1,10,22,27] or row enumeration-based [3,4,19,20], usually use the bottom-up search strategy. For the row enumeration-based algorithms, we choose *Carpenter* [19] as a representative to introduce their search strategy since they use the same kind of search strategy. Afterwards, we can introduce our top-down search strategy.

3.1. Bottom-up search strategy

Both existing column enumeration-based and row enumeration-based frequent pattern mining algorithms exploit the bottom-up search strategy as shown in Figs. 1 and 2.

Fig. 1 is a column (item) enumeration tree that shows all the item combinations. Typical frequent pattern mining algorithms such as *Apriori* [1], *FP-growth* [10] and *CHARM* [27] search for frequent patterns by traversing this tree in the breadth-first or depth-first order. Using this search order, the itemsets are traversed and checked from small to big, and we call this kind of search order **bottom-up**. When a traversed itemset is not frequent, there is no need to traverse any of its decedent nodes. Therefore, this kind of search strategy enables the effective pruning of the search space.

Fig. 2 shows a row enumeration tree that lists all the rowsets also in a bottom-up order. By traversing this tree in a depth-first or breadth-first order, the row enumeration space can be searched in an order from the small rowsets to the large ones. In this paper, since our focus is on row enumeration-based method, the bottom-up item enumeration tree will not be discussed in further detail.

In Fig. 2, each node represents a rowset, and the corresponding itemset is listed under each node. Our mining task is to discover all the large closed rowsets. Therefore, the main constraint for mining is the size of the rowset. Since it is monotonic in terms of the bottom-up search order, it is hard to prune the row enumeration search space early. For example, suppose the *minsup* is set at 3, although obviously all the nodes in the first two levels from the root cannot satisfy this constraint, these nodes still need to be checked [3,4,19,20]. As a result, as the *minsup* increases, the time needed to complete the mining process cannot decrease rapidly. This limits the application of this kind of algorithm in real situations.

In addition, the memory cost for this kind of bottom-up search is also big. Take *Carpenter* as an example. Similar to several other algorithms [3,4,20], *Carpenter* needs to save many *x*-conditional transposed tables in memory during the mining process. For a table with n rows, the maximum number of different levels of transposed tables needed to remain in memory is n , although among which of the first ($\text{minsup} - 1$) levels of pointer lists will not contribute to the final result.

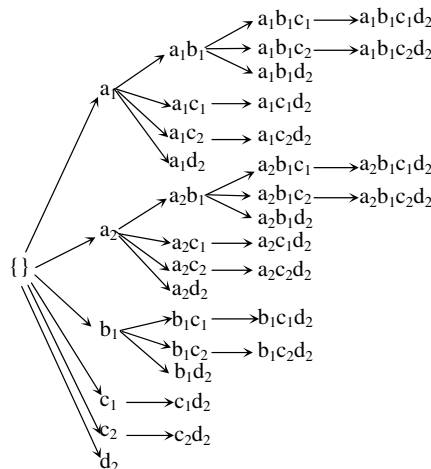


Fig. 1. Bottom-up item enumeration tree.

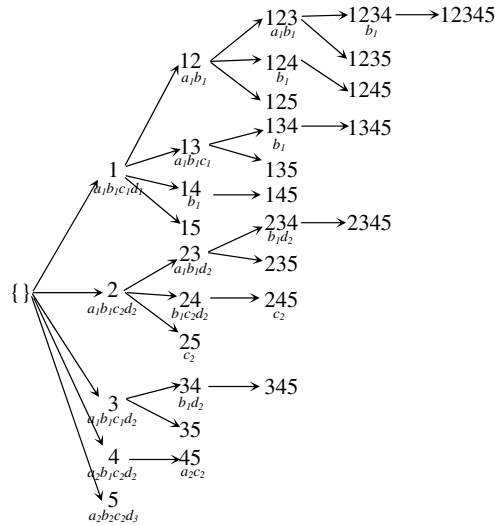


Fig. 2. Bottom-up row enumeration tree.

These observations motivate the proposal of our top-down search strategy.

3.2. Top-down search strategy

Almost all of the frequent pattern mining algorithms dealing with data set without transposition use the anti-monotonicity property of the *minsup* to speed up the mining process. For the transposed data set, the *minsup* constraint maps to the size of the rowset. To stop the search further when the *minsup* is not satisfied, we propose to exploit a top-down search strategy instead of the bottom-up. To do so, we designed a row enumeration tree following this strategy, which is shown in Fig. 3. Contrary to the bottom-up search strategy, the **top-down** search strategy means that along each search path the rowsets are checked from large to small.

In Fig. 3, each node represents a rowset, and the corresponding itemset is listed under it. We define the level of root node as 0, while the highest level for a data set with n rows is $(n - 1)$.

Lemma 3. Given a table T with n rows and the user-specified *minsup*, we can stop the further search of the top-down row enumeration tree at level $(n - \text{minsup})$ for mining frequent closed itemsets.

Proof. For each rowset at level j of the top-down row enumeration tree, its size is $(n - j)$. If $j > (n - \text{minsup})$ holds, then $(n - j) < \text{minsup}$ holds. That is to say, the size of each rowset with a level greater than $(n - \text{minsup})$ is less than the *minsup*,

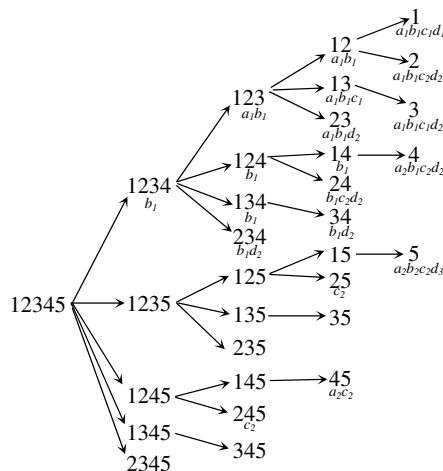


Fig. 3. Top-down row enumeration tree.

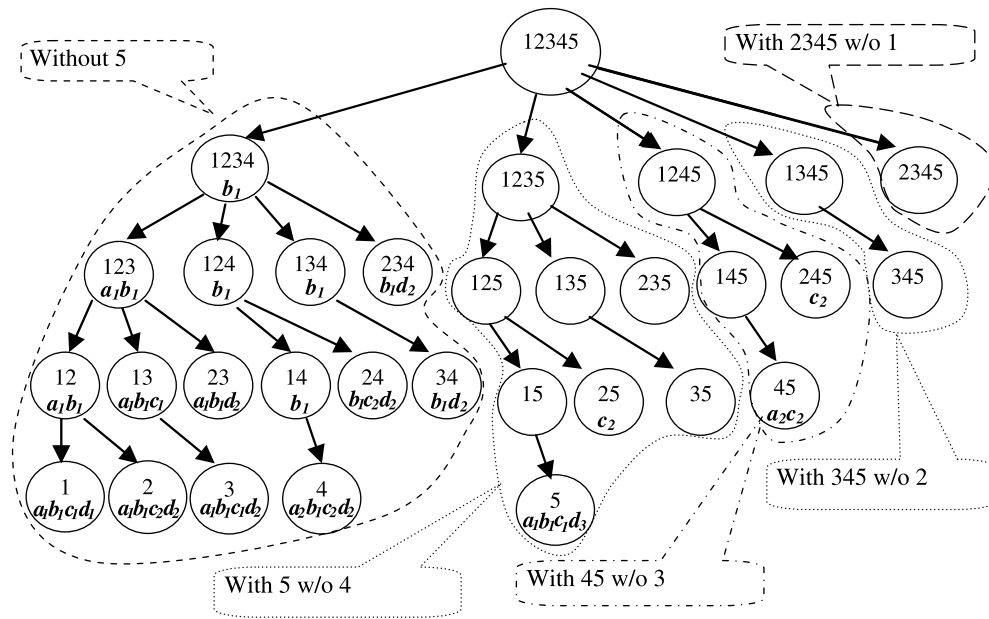


Fig. 4. Partition search space by divide-and-conquer technique.

so the support of its corresponding itemset is less than the *minsup*. Therefore, once we finish checking each rowset at level $(n - \text{minsup})$, we can stop the further checking of any rowset at the higher levels.

For example, suppose the *minsup* = 3 for the data set shown in Table 1, we can stop the further search at level 2, and the nodes at levels 3 and 4 will not contribute to the set of frequent patterns at all. □

With this row enumeration tree, we can mine the transposed table T^t by the divide-and-conquer method as shown in Fig. 4. Each node of the tree in Fig. 4 corresponds to a subtable. For example, the root represents the whole table T^t , and then it can be divided into five subtables: table without *rid* 5, table with 5 but without 4, table with 45 but without 3, table with 345 but without 2, and table with 2345 but without 1. Here, 2345 represents the set of rows {2, 3, 4, 5}, and the same holds for 345 and 45. Each of these tables can be further divided by the same rule.

We call each of these subtables ***X-excluded transposed table***, where x is a rowset that is excluded from the table. Tables corresponding to a parent node and a child node are called *parent table* and *child table*, respectively. The following is the definition of an *X-excluded transposed table*:

Definition 6 (*X-excluded transposed table*). Given a rowset $x = \{r_{i1}, r_{i2}, \dots, r_{ik}\}$ with an order such that $r_{i1} > r_{i2} > \dots > r_{ik}$ and a minimum support threshold *minsup*, an *X-excluded transposed table* $T^t|_x$ is a table in which each tuple contains *rids* less than any of the *rids* in x , and at the same time contains all the *rids* greater than any of the *rids* in x . Rowset x is called an ***excluded rowset***.

Example 6 (*X-excluded transposed table*). In the transposed table T^t shown in Table 2, two of its *X-excluded transposed tables* are shown in Tables 4 and 5, respectively, assuming the *minsup* = 2.

Table 4
X-excluded table $T^t|_{54}$.

Itemset	Rowset
a_1	1, 2, 3
b_1	1, 2, 3
c_1	1, 3
d_2	2, 3

Table 5
X-excluded table $T^t|_4$.

Itemset	Rowset
c_2	2

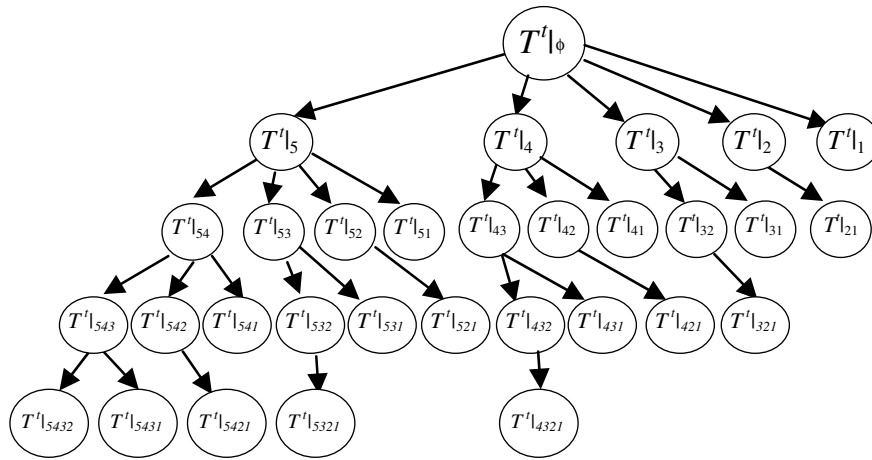


Fig. 5. Parent-child relationship between excluded transposed tables.

Table 4 shows an *X-excluded transposed table* $T^l_{|54}$, where $x = \{5, 4\}$. In this table, each tuple only contains *rids* that are less than 4, and contains at least 2 such *rids* as the *minsup* is 2. Since the largest *rid* in the original data set is 5, it is unnecessary for each tuple to contain some of the other *rids*. The procedures to derive this table are shown in Example 7.

Table 5 is an *X-excluded transposed table* $T^l_{|4}$, where $x = \{4\}$. Its parent table is the table shown in Table 2. Each tuple in $T^l_{|4}$ must contain *rid* 5 as it is greater than 4 and in the meantime must contain at least 1 *rid* less than 4 as the *minsup* is set at 2. As a result, only those tuples containing *rid* 5 can be candidate tuples of $T^l_{|4}$ in Table 2. Therefore, only tuples a_2 and c_2 satisfy this condition. However, tuple a_2 does not satisfy the *minsup* after excluding *rid* 4, so only one tuple is left in $T^l_{|4}$. Note that although the current size of tuple c_2 in $T^l_{|4}$ is 1, its actual size is 2 since it contains *rid* 5, which is not listed explicitly in the table.

The excluded transposed tables have a relationship called the parent-child relationship as shown by a tree in Fig. 5. In this tree, each node represents an excluded transposed table. For a node with children, we call it the parent table of the table represented by each of its children nodes. For example, the excluded transposed table $T^l_{|5}$ is the parent table of $T^l_{|54}$. In this tree we can easily see the parent-child relationship between the excluded transposed tables. In the meantime, it is easy to see the relationship between this tree and the tree shown in Figs. 3 and 4. The excluded rowset of each node in Fig. 5 is a complement of the rowset of its corresponding node in Fig. 3.

The *X-excluded transposed table* $T^l_{|x}$ can be obtained by the following steps:

- (1) Given a transposed table $T^l_{|p}$ which is the parent table of $T^l_{|x}$, extract from it each tuple containing all *rids* greater than r_{i1} .
- (2) For each tuple obtained in the first step, keep only the *rids* that are less than r_{ik} .
- (3) Get rid of the tuples containing less than $(\text{minsup} - j)$ *rids*, where j is the number of *rids* greater than r_{i1} in S .

The reason for the operation in step 3 will be given in Section 4. Note that the original transposed table corresponds to $T^l_{|\phi}$, where ϕ is an empty rowset.

Example 7 (Procedure to get *X-excluded transposed table*). Here we will show the steps to derive an *X-excluded transposed table* $T^l_{|54}$ from its parent table $T^l_{|5}$ in Table 6.

- (1) Each tuple in table $T^l_{|5}$ is a candidate tuple of $T^l_{|54}$ as there is no *rid* greater than 5 for the original data set.
- (2) Table 7 shows the table after excluding the *rids* not less than 4.
- (3) Since tuple c_2 only contains one *rid*, it does not satisfy the *minsup* and is thus pruned from $T^l_{|54}$. Then we get the final $T^l_{|54}$ shown in Table 4.

Table 6

Itemset	Rowset
a_1	1, 2, 3
b_1	1, 2, 3, 4
c_1	1, 3
c_2	2, 4
d_2	2, 3, 4

Table 7 $T^t_{|S_4}$ without pruning.

Itemset	Rowset
a_1	1, 2, 3
b_1	1, 2, 3
c_1	1, 3
c_2	2
d_2	2, 3

From Definition 6 and the above procedure to derive the X -excluded transposed table, it is clear that the size of the excluded table will become smaller and smaller due to the *minsup* threshold, so the search space will shrink rapidly.

Lemma 4. The minimum size threshold *minsup* is an anti-monotonic constraint for mining frequent closed itemsets from the transposed table T^t .

Proof. For a rowset S_1 with corresponding itemset I_1 , if its size is less than the *minsup* threshold (i.e., itemset I_1 is not frequent), then for a superset I_2 of I_1 , its corresponding rowset S_2 must be a subset of S_1 , which means that the size of S_2 is also less than the *minsup* threshold. Therefore, itemset I_2 is not frequent either. $\square$

4. Closeness-checking methods

Frequent closed itemset mining has two constraints: first is to check if an itemset is frequent and second is to check if it is closed. The top-down search strategy takes advantage of the first constraint to prune the search space. The method used to check if an itemset or a rowset is closed is called the *closeness-checking* method. To take advantage of the second constraint to make mining efficient, we developed a new method called the **trace-based closeness-checking** method. To compare it with the traditional closeness-checking method, we designed another method called the *output-based method* for the top-down searching strategy.

4.1. Trace-based closeness-checking method

According to Definition 4, the definition of a closed rowset for the transposed table T^t is as follows:

Definition 7 (Closed rowset in T^t). A rowset X from the transposed table T^t is closed if and only if there does not exist another rowset S_1 such that S_1 is larger than X , and it occurs in the same set of tuples in T^t .

For example, in T^t shown in Table 2, rowset $\{1, 2\}$ is not closed because there is another bigger rowset $\{1, 2, 3\}$ that occurs in the same set of tuples a_1b_1 , while rowset $\{1, 2, 3\}$ is closed since we could not find any other bigger rowset that occurs in tuples a_1 and b_1 .

The following lemmas lead to the trace-based *closeness-checking* method:

Lemma 5. Given an itemset $I \subseteq \mathcal{V}$ and a rowset $X \subseteq \mathcal{S}$, the following two equations hold:

$$r(I) = r(i(r(I))), \quad (5)$$

$$i(X) = i(r(i(X))). \quad (6)$$

Proof. Since $r(I)$ is a set of rows that shares a given set of items and $i(X)$ is a set of items common to a set of rows, these two equations are obviously correct according to this semantics and Definition 1. $\square$

Lemma 6. In the transposed table T^t , a rowset $X \subseteq \mathcal{S}$ is closed if it can be represented by an intersection of a set of tuples, that is,

$$\begin{aligned} \exists I \subseteq V, \quad \text{s.t.} \quad X = r(I) = \bigcap_j r(\{i_j\}), \\ \text{where } i_j \in I, \quad \text{and} \quad I = \{i_1, i_2, \dots, i_l\}. \end{aligned} \quad (7)$$

Proof. First, we will prove that if X is closed then $X = r(I)$ holds. According to Definitions 3 and 4 if X is closed $X = C(X) = r(i(X))$, we can always set $I = i(X)$, so $X = r(I)$ holds. Now we need to prove that if $X = r(I)$ holds then X is closed. If $X = r(I)$ holds for some $I \subseteq V$, according to Definition 3, $C(X) = r(i(X)) = r(i(r(I)))$ holds. Then based on Lemma 6 we have $r(i(r(I))) = r(I) = X$, so $C(X) = X$ holds. Therefore, X is closed. $\square$

Lemma 7. Given a rowset $X \subseteq \mathcal{S}$ in the transposed table T^t , for every tuple i_j containing X , which means $i_j \in i(X)$, if $X \neq \bigcap_j r(\{i_j\})$, where $i_j \in i(X)$, then X is not closed.

Proof. For tuple $i_j \in i(X)$, $\cap_j r(\{i_j\}) \supseteq X$ apparently holds. If $X \neq \cap_j r(\{i_j\})$ holds, then $X \subset \cap_j r(\{i_j\})$ holds, which means that there exists at least another item, say y , such that $X \cup y = \cap_j r(\{i_j\})$. So, $X \neq r(I)$, that is $X \neq r(i(X))$. Therefore, X is not closed. $\square$

Lemma 8. For the transposed table T^t , given the minsup, the tuples with the number of rid's less than the minsup will not contribute to the set of large closed rowsets.

Proof. For a large rowset X , its size must be no less than the minsup. According to Lemma 7, any closed rowset must be able to be represented as an intersection of a set of tuples. For tuples with a size less than the minsup, the intersection involving these tuples must have a size less than the minsup, so the rowset corresponding to this intersection cannot be large. $\square$

Lemmas 6 and 7 are the basis of our trace-based closeness-checking method, which is used to check whether a pattern (a rowset or an itemset) is closed. To speed up this kind of checking, we added some additional information for each X -excluded transposed table. We added the third column of the table shown in Tables 8, 9, or 10, which is called the skip-rowset. Each rowset listed in this column is called a skip-rowset, which is a set of rid's that keeps track of rid's that are excluded from the same tuple of all its parent tables. When two tuples in an X -excluded transposed table have the same rowset, they will be merged as one tuple, and the intersection of the corresponding two skip-rowsets will become the current skip-rowset.

Example 8 (skip-rowset and merging of X -excluded transposed table). In Example 7, when we obtained $T^t_{|54}$ from its parent $T^t_{|5}$, we excluded rid 4 from tuple b_1 and d_2 , respectively. The skip-rowset of these two tuples in $T^t_{|5}$ should be empty since they do not contain rid 5 in $T^t_{|\phi}$. Therefore, the skip-rowset of these two tuples in $T^t_{|54}$ is 4. Table 8 shows $T^t_{|54}$ with this kind of skip-rowsets.

In Table 8, the first two tuples have the same rowset $\{1, 2, 3\}$. After merging them, it becomes Table 9. The skip-rowset of this rowset becomes empty since the intersection of an empty set and any other set is still empty. If the intersection of skip-rowsets is empty, it means that this rowset is currently the result of the intersection of tuples containing the rowset. When it is time to output a rowset, this skip-rowset will be checked. According to Lemma 7, if it is empty, then it must be a closed rowset.

4.2. Output-based closeness-checking method

As stated in Definition 7, we developed a straightforward closeness-checking method. To speed up this checking method, we propose a new traverse order of the row enumeration tree as shown in Fig. 6.

In Fig. 6, the traverse order number is depicted under the rowset in each node. This novel traverse order is different from both the depth-first and breadth-first traverse order. We call it the **hybrid traverse order**. Each rowset is checked in this order so that all of the super rowsets will be checked before it. For example, when rowset 345 is being checked, its super rowsets 12345, 2345, and 1345 would have been checked before it, and have been put to an output set if they were closed. This way, we only need to see among its super rowsets output if there is one having the same itemset as its itemset to check

Table 8
 $T^t_{|54}$ with skip-rowset.

Itemset	Rowset	Skip-rowset
a_1	1, 2, 3	
b_1	1, 2, 3	4
c_1	1, 3	
d_2	2, 3	4

Table 9
 $T^t_{|54}$ after merge.

Itemset	Rowset	Skip-rowset
$a_1 b_1$	1, 2, 3	
c_1	1, 3	
d_2	2, 3	4

Table 10
 $T^t_{|4}$ with skip-rowset.

Itemset	Rowset	Skip-rowset
c_2	2	4

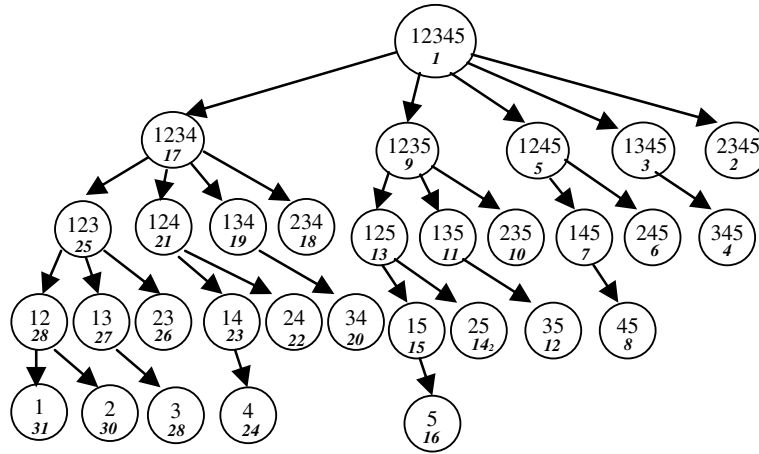


Fig. 6. Hybrid traverse order.

if a rowset is closed. If so, then it is not closed and should be discarded, otherwise, it is closed and should be put to the output set.

Lemma 9. By traversing the row enumeration tree in the hybrid traverse order, every closed rowset can be output by only checking the output set if there does not exist any of its super rowset with the same itemset.

Proof. By traversing the row enumeration tree shown in Figs. 3 or 4, every rowset will be checked. There are two possible reasons if a rowset is not put into the output set. First is that this rowset does not exist in the transposed table T^t . Second is that there is a superset of this rowset already existing in the output set, and the superset has the same itemset, which means the superset occurs in the same set of tuples. Therefore, according to Definition 7, this rowset is not closed and will not be output. Except for these two cases, the other rowsets will be output, and they must be closed since all their supersets have been checked before they are output. $\square$

Example 9 (Output-based closeness-checking method). Suppose we are now checking rowset 245, there is no output before it because rowsets 12345, 2345, 1345, 345, and 1245 do not exist in the transposed table T^t as shown in Table 2. Therefore, rowset 245 with itemset c_1 is put into the output set. Rowset 45 with itemset a_2c_2 is put to the output set because rowset 245 has a different itemset from that of rowset 45. When we deal with rowset 25 with itemset c_2 , the output set is checked and rowset 245 is found because it is the superset of rowset 25 and it has the same itemset c_2 . Therefore, rowset 25 is not closed and is discarded.

We can facilitate the process of searching the output set by making the index of the rowset in the output set. For example, when we search for the superset of rowset 25, we only need to check the rowsets that are bigger than rowset 25 and with itemsets with the same length.

This output-based closeness-checking method has a similar idea with most of the existing frequent closed pattern mining algorithms. Although the novel traverse order and index can speed up the checking process, the disadvantage of this method is that the output set needs to be checked repeatedly, which is time consuming when the output set is big. Therefore, we developed another novel method, which is the trace-based closeness-checking method.

5. Algorithms

To show the effectiveness of our top-down search strategy and the closeness-checking method, we designed two frequent closed pattern mining algorithms called **TD-Close** and **TTD-Close**, respectively to mine all the frequent closed patterns from the transposed table T^t of table T . They differ in implementation, data structure, and other pruning strategies used besides the pruning based on the *minsup* constraint.

5.1. Algorithm TD-Close

5.1.1. Major steps of TD-Close

Fig. 7 shows the main steps of the algorithm **TD-Close**. It begins with the transposition operation that transforms table T to the transposed table T^t . After the initialization of the set of frequent closed patterns FCP to empty set, the subroutine *Top-DownMine* is called to deal with each *X-excluded transposed table* and find all the frequent closed itemsets. The general processing order of the rowsets is equivalent to the depth-first search of the row enumeration tree as shown in Fig. 3.

Algorithm TD-Close**Input:** Table T , and minimum support threshold, $minsup$ **Output:** A complete set of frequent closed patterns, FCP **Method:**

1. Transform T into transposed table T^t
2. Initialize $FCP = \Phi$
3. Call $TopDownMine(T^t \setminus \phi, minsup)$

Subroutine TopDownMine($T^t \setminus \phi, cMinsup$)**Method:**

1. **Pruning P1:** if $|x| \geq (n - minsup)$ return;
2. If the size of $T^t|_k$ is 1, output the corresponding itemset if the rowset is closed, and then return.
3. **Pruning P2:** Derive $T^t|_{k \cup y}$ and $T^t|_{k'}$, where y is the largest rid among rids in tuples of $T^t|_k$,
 $T^t|_{k'} = \{ \text{tuple } t_i \mid t_i \in T^t|_k \text{ and } t_i \text{ contains } y \},$
 $T^t|_{k \cup y} = \{ \text{tuple } t_i \mid t_i \in T^t|_k \text{ and if } t_i \text{ contains } y, \text{ size of } t_i \text{ must be greater than } cMinsup \}$
 Note, we delete y from both $T^t|_{k \cup y}$ and $T^t|_{k'}$.
4. **Output:** Add to FCP itemset corresponding to each rowset in $T^t|_{k \cup y}$ with the largest size k and ending with rid k with empty *skip-rowset*.
5. Recursive call:
 TopDownMine($T^t|_{k \cup y}, cMinsup$)
 TopDownMine($T^t|_{k'}, cMinsup-1$)

Fig. 7. Major steps of the algorithm TD-Close.

The subroutine *TopDownMine* takes each *X-excluded transposed table* and the minimum support threshold $cMinsup$ as parameters and checks each candidate rowset of the *X-excluded transposed table* to see if it is closed. Candidate rowsets are large rowsets that occur at least once in table T^t . The parameter $cMinsup$ is a dynamically changing minimum support threshold as indicated in step 5 (We will explain how it changes in Example 12). There are five main steps in this subroutine, which will be explained one by one as follows:

In step 1, we apply Pruning strategy P1 to stop the processing of the current excluded transposed table.

Pruning strategy P1: If the size of the excluded itemset x , $|x|$ is equal to or greater than $(n - minsup)$, then there is no need to do further recursive calls of the *TopDownMine*. $|x|$ is the number of *rids* excluded from the current table. If it is not less than $(n - minsup)$, the size of each rowset in the current transposed table must be less than the *minsup*. Therefore, these rowsets are impossible to become large.

In step 2, if an *X-excluded transposed table* contains only one tuple, it is unnecessary to do further recursive calls to deal with its child transposed tables. So, we stop further recursive calls.

The reason for this is apparent. Suppose the rowset that corresponds to this tuple is X . From the itemset point of view, any child transposed table of this current table will not produce any different itemset from the one corresponding to rowset X . From the rowset point of view, each rowset S_i corresponding to each child transposed table of X is a subset of X , and S_i cannot be closed because $r(i(S_i)) \supseteq X$ holds, therefore $S_i \neq r(i(S_i))$ holds.

However, the current rowset X might be a closed rowset, so if the *skip-rowset* is empty we need to output it first before returning.

Example 10 There is only one tuple in table $T^t|_4$ shown in Table 10. After checking to see if this tuple is closed (apparently it is not closed because its *skip-rowset* is not empty), we do not need to produce any child excluded transposed table from it. That is, according to the excluded row enumeration tree shown in Fig. 5, all the child nodes of node {4} are pruned. This is because all the subsets of the current rowset cannot be closed any more since they are already contained by a larger rowset.

Step 3 is performed to derive from $T^t|_x$ two child excluded transposed tables $T^t|_{x \cup y}$ and $T^t|_{x'}$, where y is the largest rid among all *rids* in tuples of $T^t|_x$. These two tables correspond to the partition of the current table $T^t|_x$. $T^t|_{x \cup y}$ is the subtable without y and $T^t|_{x'}$ is the subtable with every tuple containing y . Since every rowset that will be derived from $T^t|_{x'}$ must contain y , we delete y from $T^t|_{x'}$ and at the same time decrease the $cMinsup$ by 1. Pruning strategy P2 is applied to shrink table $T^t|_{x \cup y}$ based on Lemma 8.

Pruning strategy P2: Each tuple t containing rid y in $T^t|_x$ will be deleted from $T^t|_{x \cup y}$ if the size of t (that is, the number of *rids* it contains) equals $cMinsup$.

Example 11 (Pruning strategy P2). Suppose we have currently finished dealing with table $T^t|_{54}$, which is shown in Table 9, and we need to create $T^t|_{543}$ with the $cMinsup$ being 2. According to the Pruning strategy P2, tuples c_1 and d_2 will be pruned from $T^t|_{543}$ because after excluding rid 3 from these two tuples, their size will become less than the $cMinsup$, although currently they satisfy the *minsup* threshold. As a result, there will only be one tuple $\{a_1b_1\}$ left in $T^t|_{543}$ as shown in Table 11.

Table 11 $T^t_{|543}$.

Itemset	Rowset	Skip-rowset
a_1b_1	1, 2	3

Table 12 $T^{t'}_{|54}$.

Itemset	Rowset	Skip-rowset
a_1b_1	1, 2	4
c_1	1	
d_2	2	

We can get these two tables $T^t_{|x \cup y}$ and $T^{t'}_{|x}$ by the following steps. First, for each tuple t in $T_{|x}$ containing *rid* y , delete y from it and copy it to $T^{t'}_{|x}$. Check if the size of t is less than the $cMinsup$. If not, keep it and in the meantime put y in the *skip-rowset*; otherwise, take it from $T^t_{|x}$. Finally, $T^t_{|x}$ becomes $T^t_{|x \cup y}$, and at the same time $T^{t'}_{|x}$ is obtained.

Example 12 (Tables $T^t_{|x \cup y}$ and $T^{t'}_{|x}$). For example, from table $T^t_{|54}$ we can produce two tables $T^t_{|543}$ and $T^{t'}_{|54}$ as shown in Tables 11 and 12, respectively.

Table $T^{t'}_{|54}$ is a special kind of excluded transposed table in which every tuple implicitly contains *rid* 3 (we call it common *rid*). Therefore, the minimum support threshold for the rowsets found in this table should decrease by 1, which is why in step 5 of *TD-Close* when we call the **TopDownMine** ($T^{t'}_{|x}$, $cMinsup - 1$) for Table $T^{t'}_{|54}$, the second parameter becomes ($cMinsup - 1$). That is to say, if $cMinsup$ for table $T^t_{|54}$ is 2, then the $cMinsup$ for table $T^{t'}_{|54}$ will be 1. Therefore, $cMinsup$ is a support threshold for each current transposed table, which changes dynamically during the mining process. From this table we will obtain $T^t_{|542}$, $T^t_{|541}$, and so on. We can record every common *rid* along the processing path if we want to output the rowset for each itemset.

Step 4 is the output step. If there is a tuple with an empty *skip-rowset* and the largest size, say k , containing *rid* k in $T^t_{|x \cup y}$, then output it. Since this tuple has the largest size among all the tuples in $T^t_{|x \cup y}$, there is no other tuple that will contribute to it. Therefore, it can be output. For example, in table $T^t_{|54}$ shown in Table 9, itemset a_1b_1 corresponding to rowset {1, 2, 3} can be output as its size is 3, which is larger than the size of the other two tuples, and it also contains the largest *rid* 3.

Step 5 conducts two recursive calls to deal with $T^t_{|x \cup y}$ and $T^{t'}_{|x}$, respectively.

5.1.2. Space and time analysis of *TD-Close*

When we implement the algorithm, we use the index lists to simulate the *X-excluded transposed tables*. For example, for table $T^t_{|5}$ shown in Table 6, we use two index lists $IL_{|4}$ and $IL_{|3}$ to keep the information, which are shown in Tables 13 and 14, respectively. The rowsets in index list $IL_{|j}$ all have *rid* j as the last *rid*. Since the rowset in table $T^t_{|5}$ either ended by 4 or by 3, we have two lists $IL_{|4}$ and $IL_{|3}$ for this table. For each itemset in the index list, we do not store its whole rowset, only the index of the last *rid* in the rowset. For example, for itemset “ b_1 ” in $IL_{|4}$ shown in Table 13, the last *rid* of its rowset is 4, and it is the fourth element of its rowset {1, 2, 3, 4}. Therefore, we use index 4 to record its position.

When we deal with table $T^t_{|5}$ during the mining process, we will generate another two tables: $T^t_{|54}$ and $T^{t'}_{|5}$. Taking $T^t_{|54}$ as an example, its index lists can be easily generated by following each index in $IL_{|4}$ of $T^t_{|5}$. The rowsets in $T^t_{|54}$ do not have *rid* 4; therefore, we delete 1 from the index of each itemset in $IL_{|4}$ shown in Table 13 to see its previous *rid*. For example, for itemset “ d_2 ”, we obtain the index 2 after 3 minus 1. By referring to the original transposed table in Table 2, we identified that the

Table 13Index list $IL_{|4}$ for $T^t_{|5}$.

Itemset	Index	Skip-rowset
b_1	4	5
c_2	2	
d_2	3	

Table 14Index list $IL_{|3}$ for $T^t_{|5}$.

Itemset	Index	Skip-rowset
a_1	3	
c_1	2	

Table 15Index list $IL|_3$ for $T^t|_{54}$.

Itemset	Index	Skip-rowset
a_1b_1	3	
c_1	2	
d_2	2	4

Table 16Index list $IL|_3$ for $T^t|'_5$.

Itemset	Index	Skip-rowset
b_1	3	
d_2	2	

Table 17Index list $IL|_2$ for $T^t|'_5$.

Itemset	Index	Skip-rowset
c_2	1	5

rowset of the second rid of d_2 is 3. We then add it into the index list $IL|_3$ shown in Table 14. Since we skipped rid 4 in this process, its skip-rowset will have rid 4 in it. Similarly, we put itemset “ b_1 ” in the index list $IL|_3$ and merge it with itemset a_1 as mentioned earlier. Itemset “ c_1 ” is discarded because after skipping rid 4, its rowset is left with only one rid, that dissatisfies the *minsup* 2. Now, index list $IL|_3$ becomes the only list in table $T^t|_{54}$, which is shown in Table 15. During the above process, we can get the index lists for table $T^t|'_5$ simultaneously. In this example, we will create another two index lists, which are presented in Table 16 and 17. After this, the memory used by the index list $IL|_4$ will be released.

It is easy to see that the index list becomes smaller and smaller through pruning as the mining goes on. Also, many of the index lists are reused. Furthermore, using the top-down searching style for a table T with n rows, we do not need to spend more memory for all the index lists corresponding to the nodes at levels greater than $(n - \text{minsup})$ when we stop search at level $(n - \text{minsup})$. Therefore, the biggest number of excluded transposed tables needed to remain in memory is $(n - \text{minsup} + 1)$. For a table T with m dimensions with each dimension having k distinct items (attribute values) without considering pruning, each transposed table has km distinct items in maximum. Therefore, the space complexity is about $O[km(n - \text{minsup} + 1)]$. Another part of the data that needs to remain in memory is the transposed table T^t . Without considering the merging of the items with the same rowset, the space complexity is $O(mn)$.

For time complexity, the worst case is that for a table T with n rows, each possible combination of rids exists in the transposed table T^t . In this case, *TD-Close* needs to deal with every rowset with a size not less than the *minsup*. The total number of rowsets d is $d = \sum_{k=\text{minsup}}^n \binom{n}{k}$. When dealing with each rowset, $O(1)$ time is needed to be checked if it is closed and needs to output. Then, suppose the index list with the largest rid has an average length l , $O(l)$ is needed to scan this index list and generate subsequent index lists. Suppose table T has m dimensions, and each dimension has k distinct items (attribute values), without considering pruning km distinct items distribute over n index lists, so in average there are about km/n entries in each index list. Therefore, a total of $O(dkm/n)$ time is needed to deal with the transposed table T^t . For the transformation from table T to T^t , we need $O(kmn)$ time to collect the rids for each distinct item.

5.2. Algorithm *TTD-Close*

To enhance the performance of *TD-Close* better, we developed an improved version of it, which is *TTD-Close*. Different from *TD-Close* which uses a flat table to store the transposed table, it uses a tree structure instead. This method makes the data set more compact. What is more important, it greatly changes the way of pruning. Aside from the pruning strategies used in *TD-Close*, two new pruning strategies are proposed that contribute to its better efficiency compared with the *TD-Close*.

5.2.1. Major steps of *TTD-Close*

Fig. 8 shows the main steps of the algorithm *TTD-Close*. After the transposed table T^t is generated, it will immediately be transformed into a tree structure called *FR-Tree* (Frequent Rowset Tree) with an auxiliary structure called *IP-List* (Itemset Pointer List) to keep information for the initial recursion node.

At the beginning of the algorithm *TTD-Close*, the explicit rowset in *IP-List* is assigned the complete set of rids, and the implicit rowset is assigned empty. During the recursion, any itemset in a certain *IP-List* must contain all the rids in the

Algorithm TTD-Close**Input:** Table T , and minimum support threshold, $minsup$ **Output:** A complete set of frequent closed patterns, FCP **Method:**

1. Transform T into transposed table T^t
2. Create FR -Tree and initial IP -List for T^t , in which the explicit rowset is assigned the complete set of rids and the implicit rowset is empty
3. Initialize $FCP = \Phi$
4. Call **Output** (IP -List)
5. Call **TDexclude** (IP -List)
6. Call **TDinclude** (IP -List)

Subroutine TDexclude(IP -List)**Method:**

7. **Pruning X1:** If size of explicit rowset of IP -List $\leq cMinsup$, return;
8. Create sub - IP -List by removing the largest rid from explicit rowset of IP -List
9. **Pruning X2:** Delete from explicit rowset of sub - IP -List the rids which do not exist under any itemset of sub - IP -List
10. Call **Output**(sub - IP -List)
11. If number of itemsets in sub - IP -List is 1, return
12. **Pruning X3:** If number of itemsets in sub - IP -List is 2, directly output all closed frequent sub-itemsets to FCP and return.
13. Recursive call:
 - TDexclude** (sub - IP -List)
 - TDinclude** (sub - IP -List)

Subroutine TDinclude(IP -List)**Method:**

14. **Pruning X1:** If size of explicit rowset of IP -List $\leq cMinsup$, return;
15. Create sub - IP -List by excluding the largest rid from explicit rowset of IP -List and including it in implicit rowset, and let $cMinsup = cMinsup - 1$
16. **Pruning X2:** Delete from explicit rowset of sub - IP -List the rids which do not exist under any itemset of sub - IP -List
17. If some rids are deleted in **Pruning X2**, Call **Output**(sub - IP -List)
18. If number of itemsets in sub - IP -List is 1, return
19. **Pruning X3:** If number of itemsets in sub - IP -List is 2, directly output all closed frequent itemsets to FCP and return.
20. Recursive call:
 - TDexclude** (sub - IP -List)
 - TDinclude** (sub - IP -List)

Subroutine Output(IP -List)**Method:**

Add to FCP all itemsets in IP -List with the largest rid's level equal to the size of explicit rowset and $skip$ -rowset is empty

Fig. 8. Algorithm TTD-Close.

corresponding implicit rowset and may contain some rids in the explicit rowset, excluding any rid outside these two rowsets. This will guarantee the complete traversal of all the subsets of the initial rowset based on the row enumeration tree.

Example 13 (T^t , FR -Tree, and IP -List). Fig. 9 shows the data structure of T^t , FR -Tree, and IP -List for an example data set T and the relationships among them.

The structure of FR -tree is similar to FP -tree [10]. There are two kinds of differences. First is that each node represents a rid instead of an item, and the second is that we used a parent pointer instead of a child pointer to link the nodes. Before generating this tree, rids not satisfying the $minsup$ in T^t are deleted, and the remaining rids in each tuple are ordered in the frequency descending order. This way, the size of the FR -Tree becomes more compact as more nodes can be shared.

The IP -List contains information to process each recursion node. The IP -List in Fig. 9 shows the information of the initial recursion node for the transposed table T^t that also corresponds to the node “12345” in Fig. 3. The IP -List is the implementation of the excluded transposed table that consists of four parts: a set of pointers each of which represents an itemset and points to a node in the FR -Tree, an explicit rowset, an implicit rowset, and the current minimum support threshold $cMinsup$ (which has the same meaning as the one used in TD -Close). The definition of the explicit rowset and implicit rowset is given below. Here, the implicit rowset is empty, which means every rowset is a subset of the explicit rowset $\{1, 2, 3, 4, 5\}$. Each itemset in the IP -List comes from some tuples in T^t . For example, following the pointer of itemset $\{a_1\}$, we find rid 4 first. Following the parent pointer of each node, we can find all the rids contained in the first tuple in T^t . We will use the dataset

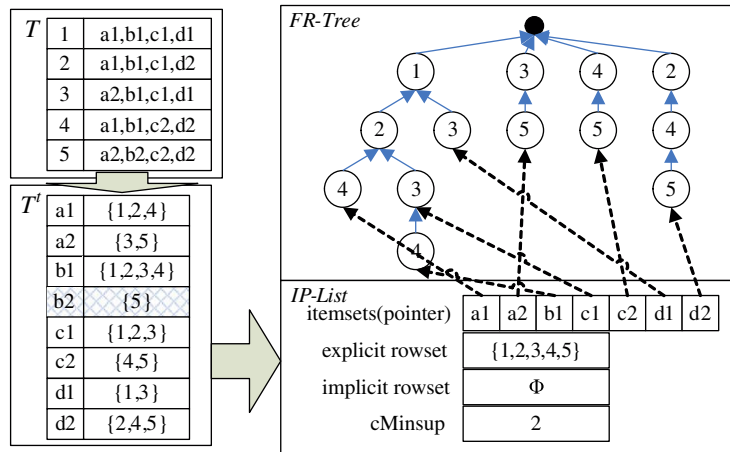
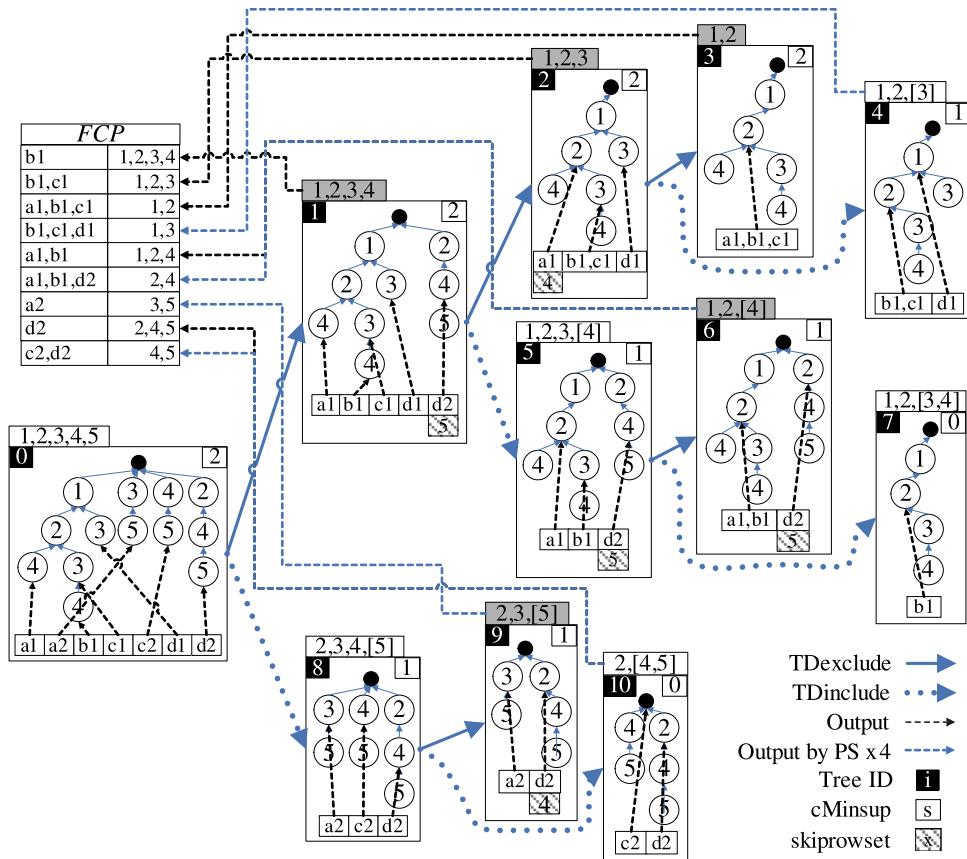
Fig. 9. Relationship among T' , FR-Tree, and IP-List.

Fig. 10. Complete recursion path.

in this example to demonstrate the algorithm *TTD-Close* in the following discussions. The complete recursion path for this dataset is shown in Fig. 10 where the trees are named by the processing order.

Definition 8 (Explicit rowset and implicit rowset). The *IP-List* is similar to the *X-excluded transposed table* where the excluded rowset *X* is the set of rid not contained in either the explicit rowset or implicit rowset. The rowset of any itemset in an *IP-List* consists of all the rid in the implicit rowset and some rid in the explicit rowset. For example, in Fig. 11b, each itemset in the

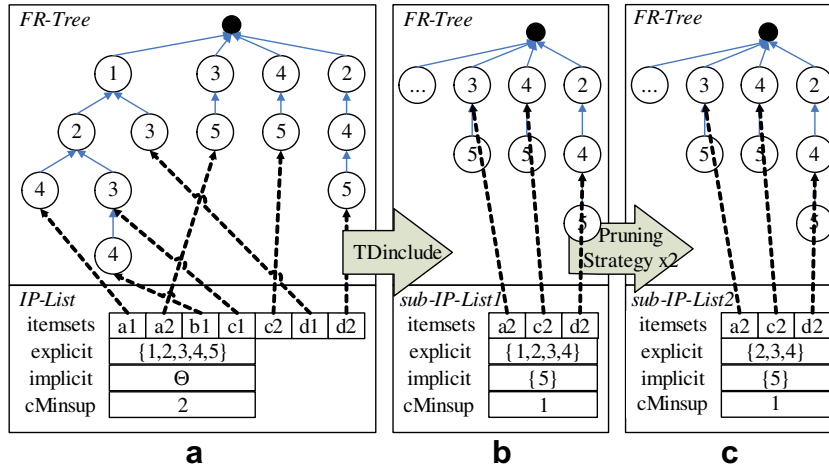


Fig. 11. Demo of Pruning Strategy X2.

sub-IP-List1 has the rowset containing rid 5 since the implicit rowset is {5}, and a sub-rowset of the explicit rowset {1, 2, 3, 4} but nothing else.

The reason that we use the explicit and implicit rowset instead of the excluded rowset X is that we can change the recursion path by changing the explicit rowset. This way, the search for some unnecessary paths can be avoided. This will be explained in detail in Pruning strategy X2.

In step 4 (row 4 in Fig. 8), after the three initialization steps in the algorithm *TTD-Close*, each itemset in the *IP-List* with the largest rid's level equal to the size of the explicit rowset is checked if it is a closed itemset. If its *skip-rowset* is empty, it is placed in the *FCP*. The subroutine *TDexclude* and *TDinclude* are then recursively called to deal with the *IP-List* and find all the frequent closed itemsets. The general processing order of rowsets is equivalent to the depth-first search of the row enumeration tree shown in Fig. 3.

In *TD-Close*, the subroutine *TopDownMine* is used to deal with both $T^t_{|x|y}$ and $T^t_{|x}$, while in *TTD-Close*, it is divided into two subroutines: *TDexclude* and *TDinclude*, because of the difference in pruning strategies. This can be considered as an alternative recursion method which follows the same path as *TopDownMine*. Take the recursion tree in Fig. 10 for illustration. Each call of *TopDownMine* corresponds to a vertex in the tree, while *TDexclude* and *TDinclude* correspond to the edges. That is why we need to call both *TDexclude* and *TDinclude* in the main algorithm as well as in each subroutine. *TDexclude* is designed to deal with $T^t_{|x|y}$, while *TDinclude* is designed to deal with $T^t_{|x}$. The main difference is that in *TTD-Close*, the *excluded transposed tables* $T^t_{|x|y}$ and $T^t_{|x}$ are in the form of *IP-Lists* instead of index lists, and all the parameters needed by the subroutines such as the *cMinsup* are contained within the *IP-List*.

There are seven main steps in the subroutine *TDexclude*, which are explained one by one as follows.

In **step 1** (row 7 in Fig. 8), we apply Pruning strategy X1 to stop the processing of the current *IP-List* when the size of the explicit rowset of *IP-List* is not greater than the *cMinsup*, which is equivalent to Pruning strategy P1 in *TD-Close*.

In **step 2** (row 8 in Fig. 8), the *sub-IP-List*, a subset of the input *IP-List*, is generated using a method similar to the generation of $T^t_{|x|y}$ in Pruning strategy P2 (step 3) in *TD-Close*. Trees 1, 2, 3, 6, and 9 in Fig. 10 adopt this method to remove the itemsets that do not satisfy the *minsup*.

In **step 3** (row 9 in Fig. 8), a new Pruning strategy X2 is used, which can change the path of recursion, therefore cutting the search space dramatically. Trees 8 and 10 in Fig. 10 use this strategy to change paths.

Pruning strategy X2: In the *sub-IP-List* generated in step 2, if a rid in the explicit rowset is not contained by any itemset, it will be removed from the explicit rowset. Thus, the recursion paths will change accordingly, which will be explained in the following example.

Example 14 (Pruning strategy X2). For the *IP-List* shown in Fig. 11a, the *sub-IP-List1* generated in step 2 is shown in Fig. 11b. Note that although the explicit rowset in the *sub-IP-List1* is {1, 2, 3, 4}, there is no “1” at all. Therefore, in step 3, the *sub-IP-List1* is further transformed into the *sub-IP-List2* shown in Fig. 11c. Simultaneously, the potential following recursion path changes from a 4-level tree to a 3-level tree as shown in Fig. 12. Fig. 12a shows the recursion path before using the Pruning strategy X2, while Fig. 12b shows the recursion path after pruning. The gray boxes show the rowsets generated by *TDexclude*, while the white boxes show those generated by *TDinclude*. In each node of these figures, the rowset inside “[]” is the implicit rowset and the outside is the explicit rowset. It is easily seen from this figure that the search space is cut dramatically by this strategy.

Lemma 10. Adopting Pruning strategy X2 will neither output any redundant frequent closed itemsets nor miss any one.

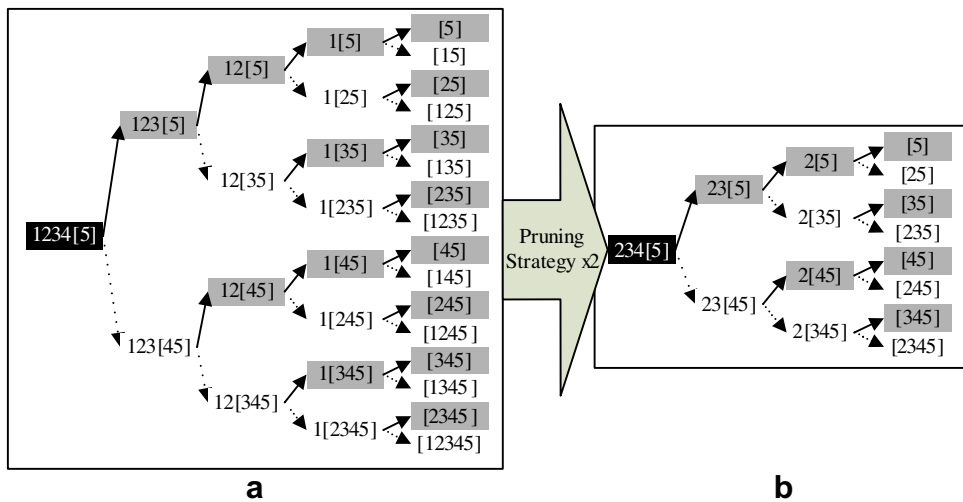


Fig. 12. Pruning Strategy X2 changes the recursion paths.

Proof. In a row enumeration tree as shown in Fig. 12a, the sub-tree under the top node (i.e., 1234 [5]) with the explicit rowset $E = \{1, 2, 3, 4\}$ and implicit rowset $I = \{5\}$ must contain all the sub-rowsets generated by adding I to every subset of E . After the activation of the Pruning strategy X2, E turns into $SE = \{2, 3, 4\}$, which is a sub-rowset of E , while I remains unchanged. Any node N' in the new sub-tree as shown in Fig. 12b will consist of I and SSE which is a subset of SE . Since SE is a subset of E , SSE must be a subset of E , thus N' must have a counterpart N that belongs to the original sub-tree. Hence, no redundant frequent closed itemsets will be found. In the meantime, all the nodes in the original sub-tree but not in the new one must contain rids pruned by the Pruning strategy X2. These nodes will never contribute to FCP according to the subroutine *Output*. Hence, no frequent closed itemsets will be missed. The new sub-tree is a compact version of the original one with no redundant frequent closed itemsets or missing ones.

In step 4 (row 10 in Fig. 8), any frequent closed itemset in the *sub-IP-List* is output using the same method as in step 4 of *TTD-close*. In Fig. 10, the table FCP gives all the frequent closed itemsets and their corresponding rowsets sorted by output order, while the dark dashed lines show where they are produced.

In step 5 (row 11 in Fig. 8), we simply stop recursion when there is only one itemset in *sub-IP-List*, which is equivalent to the step 2 in *TD-Close*. Trees 3 and 7 in Fig. 10 adopt this strategy.

In step 6 (row 12 in Fig. 8), a new Pruning strategy X3 is applied, which is an enhanced version of the one in step 5. Trees 4, 6, 9, and 10 with light dashed lines in Fig. 10 adopt this strategy. □

Pruning strategy X3: If only two itemsets remain in the *sub-IP-List*, recursion stops and the tasks left will be done directly. This can be done because it is not too complicated when only two itemsets remain.

Example 15 (Pruning strategy X3). From the *sub-IP-List2* shown in Fig. 11c, two sub-IP-Lists will be generated which are *sub-IP-List3* and *sub-IP-List4* shown in Fig. 13a. The pruning strategy X3 will be utilized when dealing with these two IP-Lists since there are only two itemsets left in both of them. For the *sub-IP-List4*, we first check the intersection of the *skip-rowsets* of these two itemsets to find if it is empty. The intersection of their rowsets is then checked, specifically, the intersection of $\{\}$ and $\{2\}$. Since the current $cMinsup$ is 0, the size of this intersection result satisfies the $cMinsup$. Therefore, the union of the two itemsets $\{c2, d2\}$ along with their common rowset $\{4, 5\}$ will be put into the FCP. Afterwards, we will check each itemset individually. $\{c2\}$ will not be put into the FCP because it has the same rowset as that of $\{c2, d2\}$, and thus it is not closed. Although $\{d2\}$ has a larger rowset and is closed, it will not be put into the FCP because its rowset contains the complete explicit rowset $\{2\}$, satisfying the output criteria of the subroutine *Output*, and it has already been put in FCP. The procedure to deal with the *sub-IP-List3* is similar. As a result, itemset $\{a2\}$ will be output. By using this pruning strategy, the change of recursion path is possible as shown in Fig. 13b.

Step 7 (row 13 in Fig. 8) conducts two recursive calls to generate and deal with the two kinds of subsets of the *sub-IP-List*.

The subroutine *TDinclude* also has seven steps, most of which are the same as that in *TDexclude* except steps 2 and 4.

In step 2 (row 15 in Fig. 8), a subset of the input *IP-List* is generated using a method similar to the generation of $T^e|_x$ in the **Pruning strategy P2** in *TD-Close*. Trees 4, 5, 7, 8, and 10 in Fig. 10 adopt this method to remove some itemsets.

In step 4 (row 17 in Fig. 8), unless some of the rids are deleted in step 3, the subroutine *Output* will not be activated. Since the *IP-List* shares the same frequent closed itemsets with its superset if the recursion path is not changed, all potential frequent closed itemsets will be output if they exist.

The subroutine *Output* will output all the itemsets containing the whole explicit rowset with the empty *skip-rowset*. The pruning strategy X1 guarantees that each itemset processed by *Output* is frequent, and the empty *skip-rowset* makes sure that

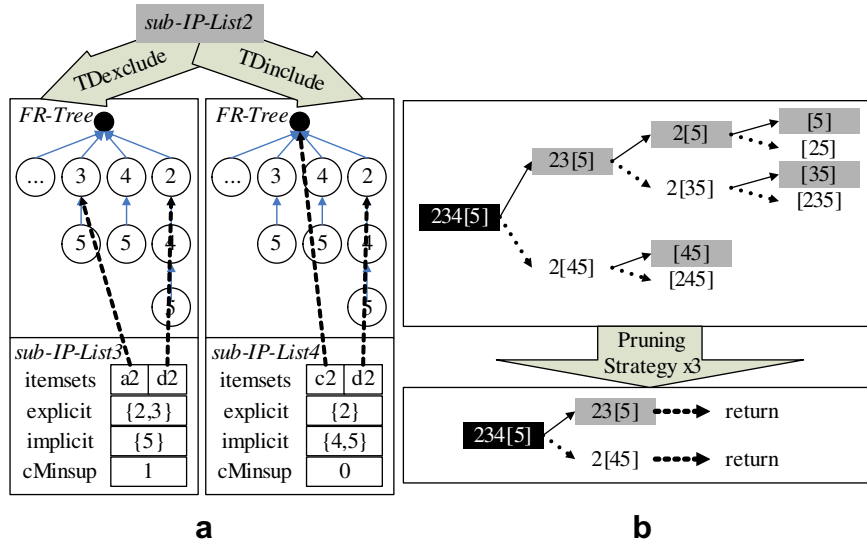


Fig. 13. Demo of Pruning Strategy X3.

the itemset is closed. Finally, since the depth-first search of the row enumeration tree can reach each possible combination of the rids once, this rule guarantees no duplicates and no loss. The subroutine *Output* is similar to the one used in *TD-Close*, but there is one main difference between them. In *TTD-close*, the condition for an itemset to be output is that the size of its row-set should equal the size of the explicit rowset. This difference is attributed to the use of the Pruning strategy X2, which makes *TTD-close* stop further recursive calls much earlier than *TD-Close*.

5.2.2. Space and time analysis of *TTD-Close*

The space complexity of *TTD-Close* is the same as *TD-Close*. Without considering the share of nodes in the *FR-tree*, the space for *FR-tree* is the same as that of the transposed table T^t which is $O(mn)$. Without considering the pruning of itemsets in each IP-list, the total space requirement is decided by both the depth of *FR-tree* in which we need to process, and the space needed by each recursion node. Given the minimum support threshold *minsup*, the biggest depth we need to process is $(n - \text{minsup} + 1)$. For each recursion node, the maximum number of itemsets is km , where k is the average number of distinct items in each dimension of T . Therefore, the total space is about $O[km(n - \text{minsup} + 1)]$.

The total time complexity during the mining process is decided by both the number of recursion nodes and the processing time taken by each node. Without considering any pruning strategy, the total number of recursion nodes is $d = \sum_{k=n}^{\text{minsup}} \binom{n}{k}$. In each recursion node, *TTD-Close* needs to process each itemset with $O(1)$ time, thus a total of $O(km)$ at most. Therefore, we get quite a loose bound of time complexity $O(dkm)$. In the transformation from T to T^t , the algorithm needs to collect rids for each item. Specifically, for each row and each dimension, *TTD-Close* needs to collect the rid of that row into one of the k items in that dimension, which will take $O(kmn)$. In the process of transformation from T^t to *FR-Tree*, the time complexity is at most $O[mn(n - 1)]$ because there are at most (mn) rids in T^t and each rid in *FR-Tree* has at most $(n - 1)$ children to be searched (specifically, the rid on the i th level has at most $(n - i)$ children). Since $(n - 1)$ cannot be smaller than k , the total time complexity should be $O[mn(n - 1)]$.

From this analysis, it seems that *TTD-Close* has a higher time complexity than *TD-Close*. However, since *TTD-Close* uses more pruning strategies than *TD-Close*, which is hard to take into consideration when analyzing time complexity, *TTD-Close* actually outperforms *TD-Close* in both real and synthetic datasets.

6. Experimental study

In this section, we will study the performance of the algorithms *TD-Close* and *TTD-Close* by comparing them with two other algorithms *Carpenter* and *FPclose*. *Carpenter* is a representative of the existing row enumeration-based algorithms since it has demonstrated its much better performance compared with the column enumeration-based algorithms such as *CLOSET* [22] and *CHARM* [27]. *FPclose* is a representative of the column enumeration-based algorithms, which won the *FIMI'03* best implementation award [7].

For the algorithm *FPclose*, we downloaded the source of the implementation from the FIMI repository [8].

For the algorithm *Carpenter*, we implemented it to the best of our knowledge according to paper [19] and its subsequent papers [3,4], [20]. We also improved it by using backward pruning, which is a faster *closeness-checking* method used in

several other algorithms dealing with *microarray data* [4,19]. The original *closeness-checking* method can be described as follows: before outputting each itemset found currently, we must check if it has been already found before. If not, output it. Otherwise, discard it. This method is slower than the backward pruning method because the itemset from very high dimensional data set usually contains a large number of items, and it is not in a specific order, so comparison with a large number of large itemsets takes a long time. This is also the reason why some of the algorithms proposed after *Carpenter* use backward checking method. However, when dealing with every rowset, *Carpenter* with backward pruning strategy still needs to check the corresponding conditional transposed table to find the largest common rowset. It also needs to check the original transposed table to do backward pruning.

This set of experiments was performed on a personal computer with a Pentium-4 1.5 GHz CPU, 1 GB RAM, and 30 GB hard disk. All of the runtimes plotted in the figures included both computation time and I/O time. Experiments for both synthetic data sets and real *microarray data* sets were conducted.

6.1. Synthetic data sets

Synthetic data sets are used to test performance in different aspects such as the effect of the change of *minsup*, the number of dimensions, the number of tuples, the cardinality of each dimension, and the skew factor on the runtime.

Synthetic data sets are generated randomly based on four main parameters: the number of dimensions, the number of tuples, the number of different values in each dimension, and the skew factor. We use **D#T#C#S#** to represent the synthetic dataset, where D# is the number of dimensions, T# is the number of tuples, C# is cardinality, and S# is the skew factor or zipf of the data. When S# equals 0, the data is uniform. The bigger the S#, the skewer is the data. The skew factor is applied to all the dimensions in a particular data set.

6.1.1. Closeness-checking methods

We first tested the effectiveness of the *closeness-checking* method developed in this paper. We compared the algorithm **TD-Close** using the *trace-based closeness-checking*, with **TD-Close** using the traditional output-based *closeness-checking* method that checks the frequent itemset output to see if an itemset is closed. We call the latter algorithm **TD-Close-O**. In this set of experiments, the synthetic dataset is D2000T100C10S0. Fig. 14 shows the comparison results.

Fig. 14 shows the runtime of the two algorithms with different *minsup* values. As we can see from this figure, algorithm **TD-Close** outperforms **TD-Close-O** especially when the *minsup* becomes small. This is because when the *minsup* becomes smaller, the number of large closed rowsets becomes bigger. Whenever we need to judge if a large rowset is closed, the output set will be checked, and therefore, more time is needed when the *minsup* becomes smaller. Similar results can be obtained when we change the four parameters of the data sets.

From this set of experiments, we can conclude that the *trace-based closeness-checking* method is more efficient than the traditional output-based *closeness-checking* method.

6.1.2. Dimensionality

To test the performance of the four algorithms with respect to the number of dimensions, we generated four data sets with 4000, 6000, 10,000, and 20,000 dimensions, respectively. Fig. 15a–d shows the effect of changing the dimensionality on the runtime of the four algorithms.

In this set of experiments, *Carpenter* is not able to finish in several hours even when the constraint *minsup* is relatively high. We can see from Fig. 15a that as the *minsup* decreases, the runtime of these three algorithms increases. Apparently, the increase speed of the algorithm **TTD-Close** and **TD-Close** is not fast, while when the *minsup* reaches 11, the runtime of **FPclose** increases dramatically. This is because when the *minsup* reaches 11, the number of frequent single items increases dramatically. Since **FPclose** is a column enumeration-based algorithm, the increase of the number of frequent items will lead

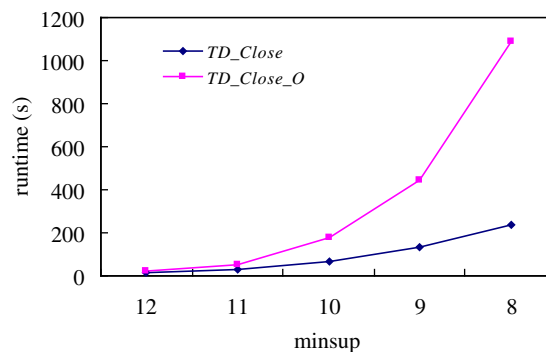


Fig. 14. Runtime for D2000T100C10S0.

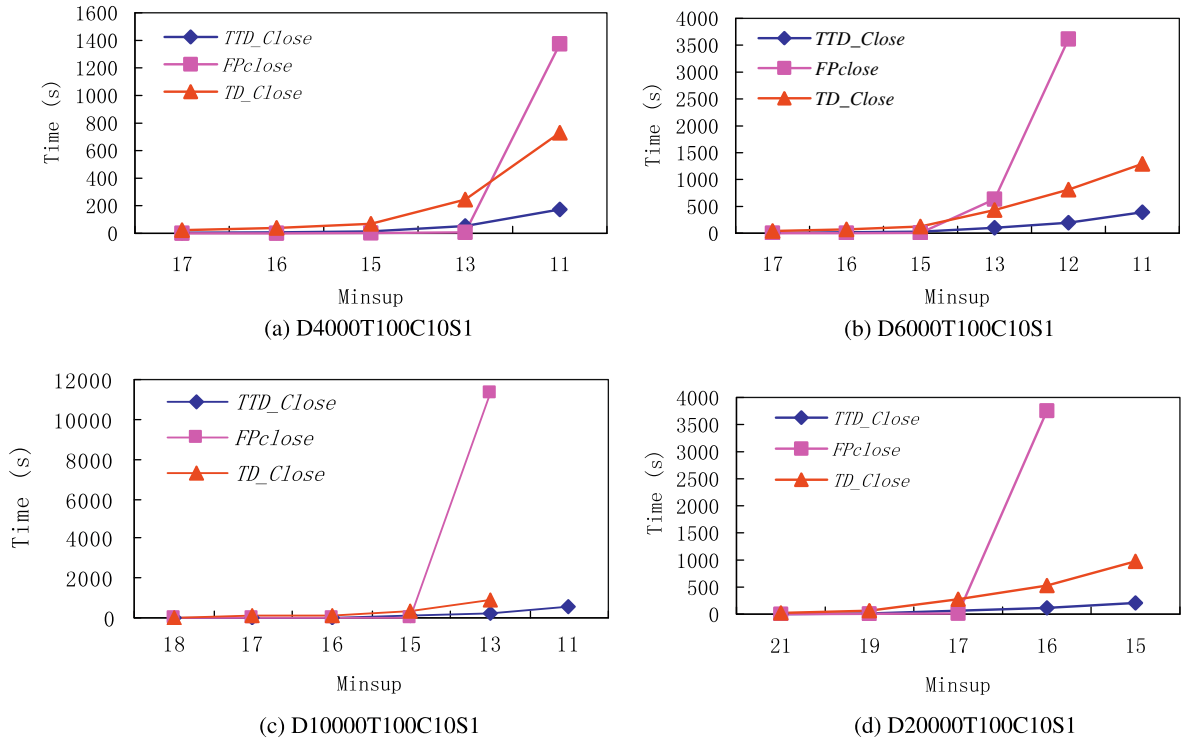


Fig. 15. The effect of changing the dimensionality on the runtime.

to the explosion of the number of frequent itemsets that need checking. On the other hand, the row enumeration-based algorithms such as *TD-Close* and *TTD-Close* search the row combination space. The number of rows influences the runtime of these algorithms more than the number of frequent items. When the *minsup* is high, they take a similar amount of time. However, when the *minsup* becomes low, *TTD-Close* and *TD-Close* perform better than *FPclose*, while *TTD-Close* performs better than *TD-Close*.

Similar comparison results can also be seen in Fig. 15b–d. Furthermore, one can also see that all of these algorithms need more time for data sets with more dimensions for the same *minsup* value, and this is due to the rise of the number of itemsets as the number of dimensions increases. Data sets with a dimensionality larger than 20,000 show the same results for these algorithms.

6.1.3. Number of tuples

To test the runtime of these algorithms with respect to the number of tuples, two more data sets are produced. One contains 150 tuples and the other 200 tuples, while the dimension is 4000, cardinality is 10, and the skew factor is 1. Fig. 16a–c shows the experimental results. To obtain a convenient comparison, we put the results of D4000T100C10S1 here again.

For this set of experiments, *Carpenter* is not able to run to completion due to the error of running out of memory. As for the other three algorithms, we can see that for some relative high values of the *minsup*, *FPclose* runs a little faster than *TD-Close* and *TTD-Close*. However, when the *minsup* becomes relatively small, both *TD-Close* and *TTD-Close* run faster than *FPclose*. The reason for this is the same as the explanation above. As for *TD-Close* and *TTD-Close*, it is easy to see that the latter performs better than the former. This set of experiments also gives us a trend: as the number of tuples increases, *FPclose* performs relatively better. This is easy to understand because *FPclose* is less influenced by the number of tuples compared with *TD-Close* and *TTD-Close*.

6.1.4. Cardinality

In the above groups of experiments, the cardinality of each data set is set at 10, which means that each dimension of each dataset has 10 distinct values. To test the performance of the different cardinalities, two more datasets are generated that correspond to 12 distinct values and 8 distinct values. The other parameters remain unchanged, that is to say, the number of dimension is 4000, the number of tuples is 100, and the skew factor is 1. The experimental results are shown in Fig. 17a–c.

The results of *Carpenter* are not shown in the first two data sets due to the same reason explained above. Fig. 17a and b tells us the same story shown in Figs. 15 and 16. Fig. 17c shows that when the *minsup* becomes small, less than 10 for example, *TTD-Close*, *TD-Close*, and *Carpenter* spend less time than *FPclose*. Compared with the data set D4000T100C12S1, data set D4000T100C8S1 is relatively denser. With the same *minsup* threshold, the latter data set has more frequent patterns.

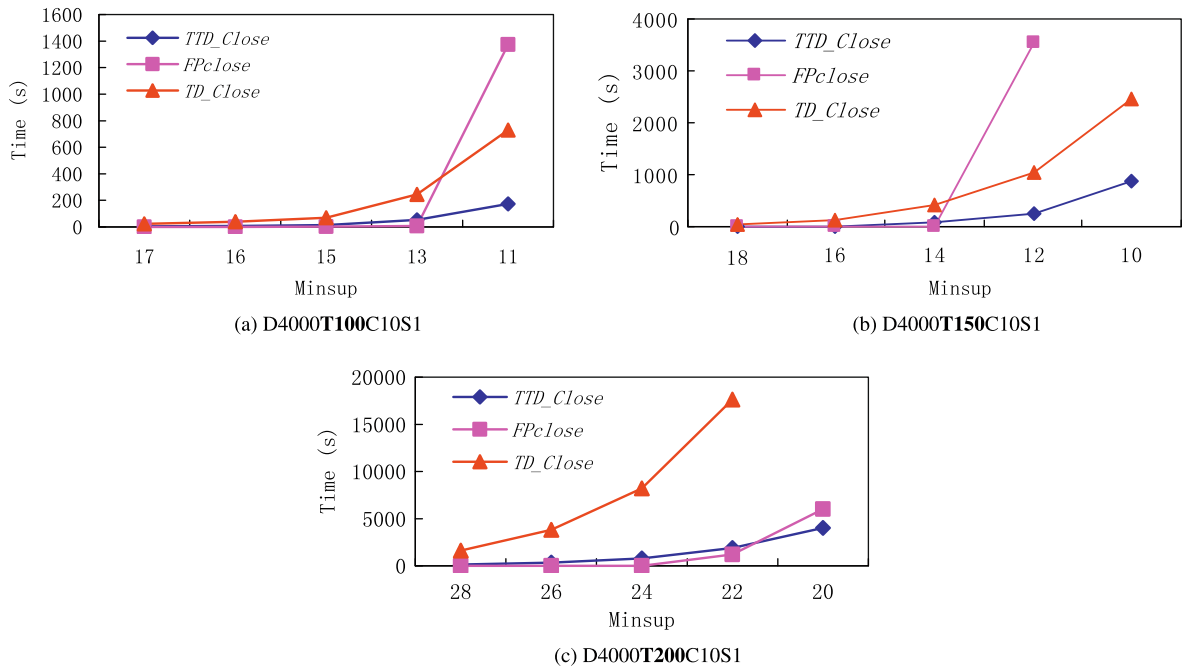


Fig. 16. The effect of changing the number of tuples on the runtime.

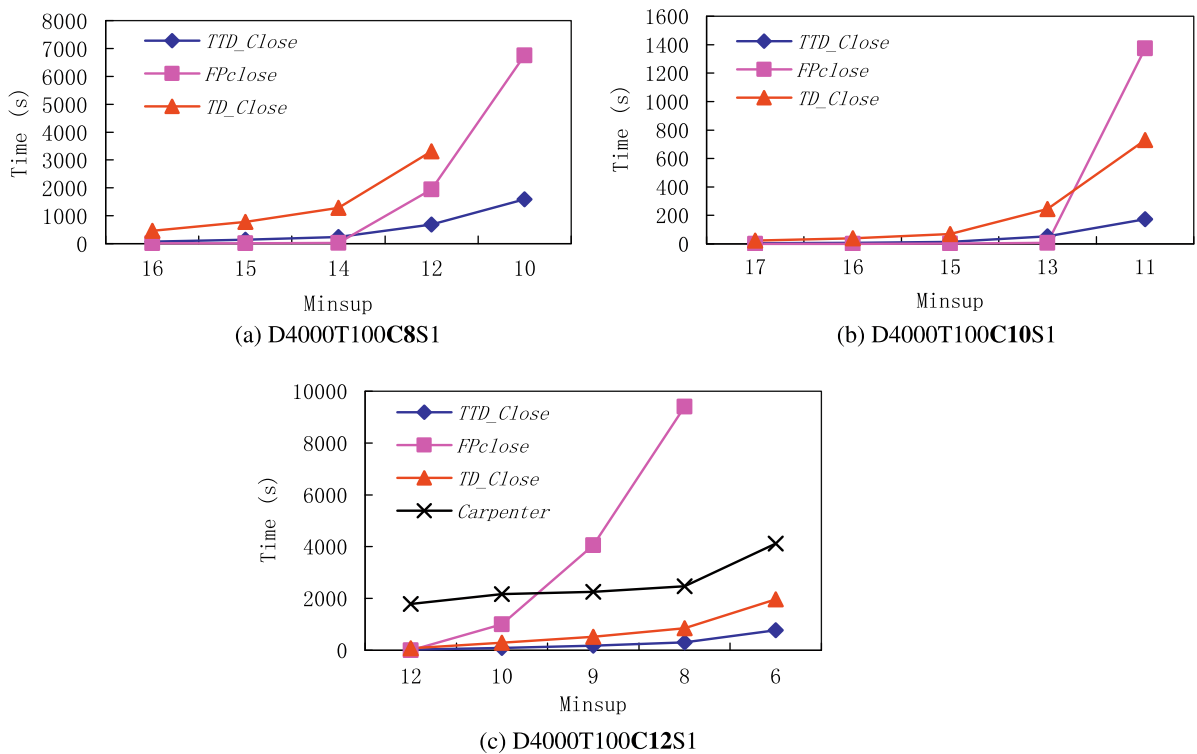


Fig. 17. The effect of changing cardinality on the runtime.

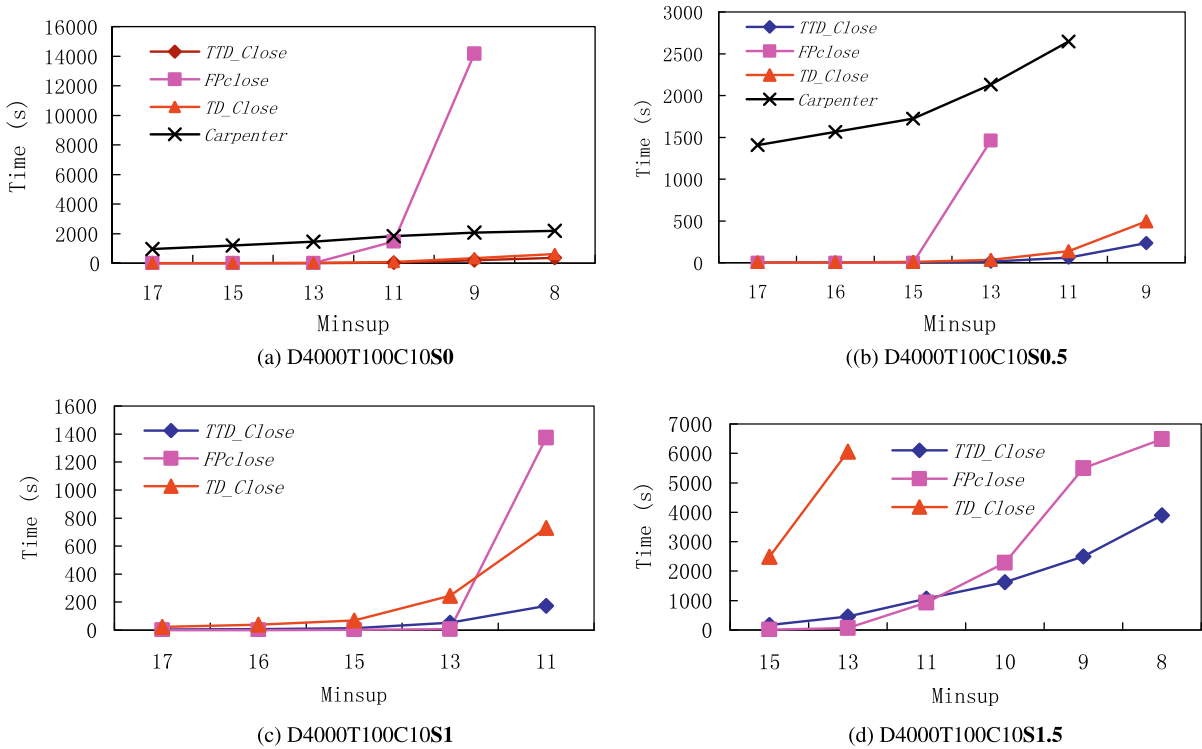


Fig. 18. The effect of the change of skew factor on the runtime.

Therefore, it took more time than the former data set at the same *minsup* threshold. This is also the reason why *Carpenter* is not able to finish in several hours for the first two data sets. Furthermore, these figures also indicate that *Carpenter* performs relatively better when the data set become sparse.

6.1.5. Skew factor

To test the performance of these algorithms with respect to the skew factor of the data set, three new data sets are generated with a skew factor 0, 0.5, and 1.5, respectively, while the dimension, cardinality, and the number of tuples remain 4000, 100, and 10. Fig. 18a–d shows the experimental results.

From Fig. 18a and b, one can see that *TTD-Close* performs the best among the four algorithms, while *TD-Close* is the second best. With a lower *minsup* threshold, *Carpenter* is better than *FPclose*. As the data set becomes skewer, *Carpenter*'s performance become worse as it cannot run to the end successfully in several hours on the other two datasets. On the contrary, *FPclose* becomes better when the skew factor increases as shown in Fig. 18a–d. The reason *Carpenter* performs relatively better on less skew data is that when data are more uniform, the pruning power of the *minsup* is weaker as almost every item has the same frequency. When the *minsup* is not big, most of the itemsets will not be pruned. Therefore, the advantage of the algorithms *TTD-Close*, *TD-Close*, and *FPclose* cannot be fully used. On the other hand, *FPclose* can make good use of the skew characteristic of the dataset to filter out infrequent itemsets early. Therefore, it performs relatively better when data is skewer. Comparing these figures, we can also see that the difference between *TTD-Close* and *TD-Close* becomes bigger when data become skewer. The enhancement of *TTD-Close*'s performance is due to the new Pruning strategy *X2* used. Overall, *TTD-Close* gives the best performance among the four algorithms in this set of experiments.

To sum up, from the experimental results conducted on different synthetic data sets, *TTD-Close* performs the best among the four algorithms in most cases, especially when the *minsup* is low. In addition, *TTD-Close*'s performance is at least an order of magnitude better than *TD-Close*.

6.1.6. Memory cost

To compare the memory usage of these algorithms, we monitored the largest amount of memory used by each algorithm. Fig. 19 shows the amount of memory used for the dataset with 4000 attributes, 100 tuples, a cardinality of 10, and a skew factor of 0.5. Note that the y-axes in this figure is in a logarithmic scale.

From Fig. 19, we can see that *TTD-Close* and *TD-Close* use less memory, while *FPclose* and *Carpenter* use more memory. The reason why *Carpenter* uses more memory is that it spends more memory in dealing with the rowsets smaller than the *minsup*. *FPclose* spends more memory on more *fp-trees* and other auxiliary data structures. A comparison on other datasets reveals similar results.

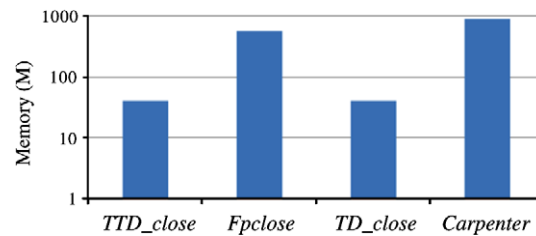


Fig. 19. Memory usage comparison.

6.2. Real data sets

Except for the synthetic data sets, we also tested the algorithms on three real *microarray data* sets: clinical data on ALL-AML leukemia, lung cancer, and breast cancer [3,19]. We took the corresponding training data sets for the experiments. ALL-AML has 38 rows and 7129 dimensions, lung cancer has 32 rows and 12,533 dimensions, and breast cancer has 78 rows and 24,481 dimensions. Finding frequent itemsets from these datasets is very useful. In these cases, each item is a gene. Frequent itemsets mean that they occur together frequently. These itemsets can be used to build a classifier to help diagnose disease. Before using the frequent pattern mining algorithms, they are all discretized using the equal-frequency binning method with five bins for each dimension.

The figures from Figs.20–22 show the runtime of TD-Close, TTD-Close, Carpenter, and FPclose at different *minsup* values for the three data sets, respectively. Note that the y-axes in these figures are in a logarithmic scale, and we plot the figures as the *minsup* increases so that we can see the pruning power of the *minsup* clearly.

Fig. 20 shows that as the *minsup* increases, the runtime of TTD-Close, TD-Close, and FPclose reduces dramatically, while the runtime of Carpenter remains relatively stable. This is because Carpenter still needs to search the rowset space in which the size of each rowset is less than the *minsup*, although apparently they will not satisfy the *minsup*. This figure also indicates

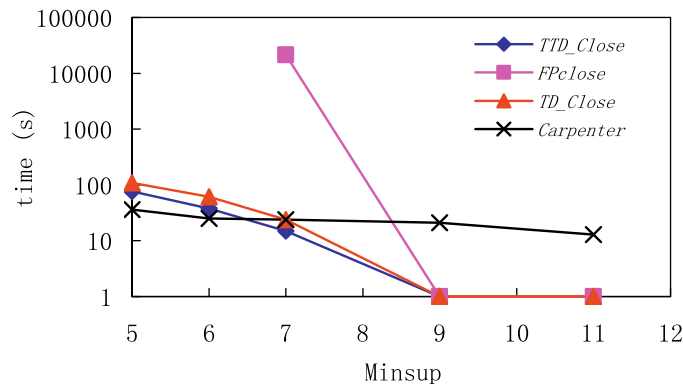


Fig. 20. Runtime on ALL-AML leukemia.

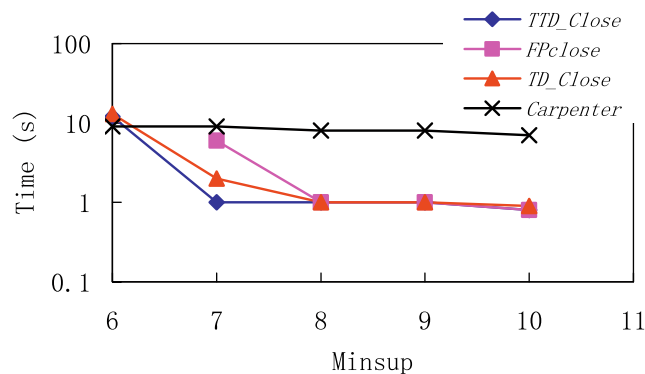


Fig. 21. Runtime on lung cancer.

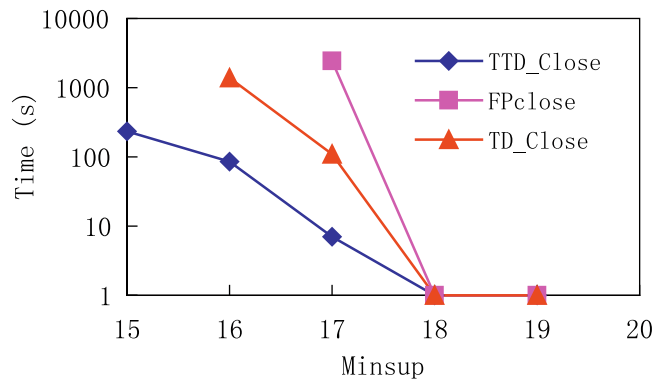


Fig. 22. Runtime on breast cancer.

that *Carpenter* is a little bit faster than *TTD-Close* when the *minsup* is small because a low *minsup* means there is small room for pruning, and *TTD-Close* cannot benefit much from this kind of pruning. Another reason is that since the cardinality is 5, the data set is relatively dense. However, the difference between them is not big. We did not get the runtime of *FPclose* when the *minsup* was less than 7 because when the *minsup* is 7, it has already spent a longer time than the other algorithms.

Fig. 21 shows us almost the same situation as Fig. 20. What is different is that *FPclose* cannot run successfully when the *minsup* is 6 due to a memory error. This is because the number of dimensions is large for this data set. When the *minsup* is 6, the number of itemsets that need checking is huge. For the row enumeration algorithms *TTD-Close*, *TD-Close*, and *Carpenter*, they only check the rowsets instead of itemsets, and the number of rowsets does not change dramatically.

For the breast cancer data set, we did not get the runtime for *FPclose* when the *minsup* was less than 17 due to memory error (memory ran out). Similarly, *Carpenter* cannot run to completion when the *minsup* is 19 after about five hours of running. Fig. 22 tells us that among the three algorithms, *TTD-Close* is better than *TD-Close*, while *TD-Close* is better than *FPclose*.

Overall, the results of these experiments for the real world *microarray* data sets illustrate that *TTD-Close*, *TD-Close*, and *FPclose* can use the anti-monotonic constraint *minsup* to prune the search space dramatically, while *Carpenter* cannot. Since the pruning strategies used in *TD-Close* and *TTD-Close* benefit from the top-down search strategy, we can conclude that this search strategy is more effective and useful compared with the bottom-up search strategy for row enumeration-based search algorithms. Also, for very high dimensional datasets, the row enumeration-based algorithms *TTD-Close* and *TD-Close* outperform the column enumeration-based algorithm *FPclose*.

7. Related work

A very high dimensional data set usually consists of thousands of columns but only a small number of rows. *Microarray* data is a typical example of such data. Finding interesting patterns from such data has many applications; therefore, designing efficient algorithms for it has attracted substantial interest from researchers. Traditionally, many algorithms [1,6,10,16,17,21,22,27] have been proposed to find association rules or association-based classification rules from not very high dimensional data sets such as transactional data sets. Suppose there are n different items in the data set, these algorithms usually adopt a bottom-up strategy to search the itemset space as large as 2^n . Through a bottom-up search, these algorithms can use the *minsup* threshold to stop the further search of the superset of an itemset once this itemset does not satisfy the *minsup* threshold. However, for high dimensional data sets, since n is large, the search space becomes huge. Therefore, these algorithms suffer from poor performance in high dimensional data. To solve this problem, a new method called transposition was first proposed in [24] for pattern discovery by doing a transposition of the original data set. Afterwards, a group of algorithms [3,4,19], [20] were proposed to deal with the long *microarray* data. Our work is directly related to these algorithms. In [19], an algorithm called *Carpenter* was proposed to find all the frequent closed itemsets. *Carpenter* conducts a depth-first order traversal of the row enumeration tree as shown in Fig. 2 and checks each rowset corresponding to the node visited to see if it is large and closed. In [3], an algorithm called *Farmer* was developed to find all association-based classification rules. *Farmer* also searches the row enumeration tree by depth-first order. The difference is that at each node it outputs, the association rules satisfy more constraints such as confidence. In [20], the algorithm *Cobbler* was proposed to find frequent closed itemsets by integrating the row enumeration-based method with the column enumeration-based method. Its high performance is evident when conducting experiments on a data set with a high dimension and a relatively large number of rows. In [4], an algorithm was proposed to find the top- k classification rules for each row of the *microarray* data set. The difference between this work and *Farmer* is that it stops further searching once the k most confident rules have been found for all rows. All of these algorithms aim to facilitate the mining of frequent patterns by searching the row enumeration space, and they all search the space in a bottom-up style.

8. Conclusions

In this paper, we propose a top-down search technique and a novel top-down row enumeration tree to mine frequent closed patterns from very high dimensional data based on the transposition method. We also developed a new *closeness-checking* method to check if an itemset is closed. Analysis and comprehensive experimental study show that the top-down search style can take advantage of the minimum support constraint to cut down the row enumeration search space, while the trace-based *closeness-checking* method is efficient for *closeness-checking*.

Future work includes studying how to use the top-down search strategy for mining other kinds of interesting patterns such as association-based classification rules. In addition, integrating the row enumeration-based method and column enumeration-based method in a top-down search style may be easier than in the bottom-up style. Therefore, more research can also be done with regard to this.

References

- [1] R. Agrawal, R. Srikant, Fast algorithms for mining association rules, in: J.B. Bocca, M. Jarke, C. Zaniolo (Eds.), Proceedings of the 1994 International Conference on Very Large Data Bases (VLDB'94), San Francisco, CA, USA, 1994, pp. 487–499.
- [2] Y. Cheng, G.M. Church, Biclustering of expression data, in: Philip E. Bourne, Michael Gribskov, Russ B. Altman, Nancy Jensen, Debra A. Hope, Thomas Lengauer, Julie C. Mitchell, Eric D. Scheeff, Chris Smith, Shawn Strande, Helge Weissig (Eds.), Proceedings of the 8th International Conference on Intelligent Systems for Molecular Biology (ISMB 2000), San Diego, California, USA, August 19–23, vol. 8, 2000, pp. 93–103.
- [3] G. Cong, A.K.H. Tung, X. Xu, F. Pan, J. Yang, FARMER: finding interesting rule groups in microarray datasets, in: G. Weikum, A.C. Konig, S. Deßloch (Eds.), Proceedings of the 23rd ACM International Conference on Management of Data, Maison de la Chimie, Paris, France, June 13–18, 2004, pp. 143–154.
- [4] G. Cong, K.-L. Tan, A.K.H. Tung, X. Xu, Mining top-*k* covering rule groups for gene expression data, in: Fatma Özcan (Eds.), Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14–16, 2005, pp. 670–681.
- [5] C. Creighton, S. Hanash, Mining gene expression databases for association rules, Bioinformatics 19 (1) (2003) 79–86.
- [6] G. Dong, X. Zhang, L. Wong, J. Li, CAEP: classification by aggregating emerging patterns, in: Setsuo Arikawa, Koichi Furukawa (Eds.), Discovery Science, Second International Conference, DS'99, Tokyo, Japan, December 1999, pp. 30–42.
- [7] G. Grahne, J. Zhu, Efficiently using prefix-trees in mining frequent itemsets, in: B. Goethals, M.J. Zaki (Eds.), Proceedings of the IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI 2003), CEUR Workshop Proc., vol. 90, 2003. <<http://CEUR-WS.org/Vol-90/>>.
- [8] G. Grahne, J. Zhu, Efficiently using prefix-trees in mining frequent itemsets. [Online] Source code available: <<http://fimi.cs.helsinki.fi/src/>>.
- [9] Jiawei Han, Hong Cheng, Dong Xin, Xifeng Yan, Frequent pattern mining: current status and future direction, Data Mining and Knowledge Discovery 15 (1) (2007) 55–86.
- [10] J. Han, J. Pei, Mining frequent patterns by pattern growth: methodology and implications, SIGKDD Explorations 2 (2000) 14–20.
- [11] Tianming Hu, Sam Yuan Sung, Hui Xiong, Qian Fu, Discovery of maximum length frequent itemsets, Information Sciences 178 (1) (2008) 69–87.
- [12] Anthony J.T. Lee, Chun-Sheng Wang, An efficient algorithm for mining frequent inter-transaction patterns, Information Sciences 177 (17) (2007) 3453–3476.
- [13] Anthony J.T. Lee, Ruey-Wen Hong, Wei-Min Ko, Wen-Kwang Tsao, Hsiu-Hui Lin, Mining spatial association rules in image databases, Information Sciences 177 (7) (2007) 1593–1608.
- [14] Yue-Shi Lee, Show-Jane Yen, Incremental and interactive mining of web traversal patterns, Information Sciences 178 (2) (2008) 287–306.
- [15] J. Li, L. Wong, Identifying good diagnostic genes or genes groups from gene expression data by using the concept of emerging patterns, Bioinformatics 18 (2002) 725–734.
- [16] W. Li, J. Han, J. Pei, CMAR: accurate and efficient classification based on multiple class-association rules, in: Nick Cercone, Tsau Young Lin, Xindong Wu (Eds.), Proceedings of the 2001 International Conference on Data Mining (ICDM'01), San Jose, CA, USA, 2001, pp. 369–376.
- [17] B. Liu, W. Hsu, Y. Ma, Integrating classification and association rule mining, in: Rakesh Agrawal, Paul E. Stolorz, Gregory Piatetsky-Shapiro (Eds.), Proceedings of the Fourth International Conference on Knowledge Discovery and Data Mining (KDD-98), New York City, New York, USA, 1998, pp. 80–86.
- [18] C. Niehrs, N. Pollet, Synexpression groups in eukaryotes, Nature 402 (1999) 483–487.
- [19] F. Pan, G. Cong, A.K.H. Tung, J. Yang, M.J. Zaki, CARPENTER: finding closed patterns in long biological datasets, in: L. Getoor, T.E. Senator (Eds.), Proceedings of the 2003 ACM SIGKDD International Conference On Knowledge Discovery and Data Mining, Washington, DC, USA, 2003, pp. 637–642.
- [20] F. Pan, A.K.H. Tung, G. Cong, X. Xu, COBBLER: combining column and row enumeration for closed pattern discovery, in: Proceedings of the 2004 International Conference on Scientific and Statistical Database Management (SSDBM'04), Santorini Island, Greece, June 2004, pp. 21–30.
- [21] N. Pasquier, Y. Bastide, R. Taouil, L. Lakhal, Discovering frequent closed itemsets for association rules, in: C. Beeri, P. Buneman (Eds.), Proceedings of the 7th International Conference on Database Theory (ICDT'99), Jerusalem, Israel, 1999, pp. 398–416.
- [22] J. Pei, J. Han, R. Mao, CLOSET: an efficient logarithm for mining frequent closed itemsets, in: D. Gunopulos, R. Rastogi (Eds.), Proceedings of the 2000 ACM-SIGMOD International Workshop Data Mining and Knowledge Discovery (DMKD'00), Dallas, TX, May 2000, pp. 11–20.
- [23] Tieyun Qian, Hui Xiong, Yuanzhen Wang, Enhong Chen, On the strength of hyperclique patterns for text categorization, Information Sciences 177 (19) (2007) 4040–4058.
- [24] F. Rioult, J. Boulicaut, B. Cremilleux, J. Besson, Using transposition for pattern discovery from microarray data, in: Mohammed Javeed Zaki, Charu C. Aggarwal (Eds.), Proceedings of the 8th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery, San Diego, California, USA, 2003, pp. 73–79.
- [25] J. Yang, H. Wang, W. Wang, P.S. Yu, Enhanced biclustering on gene expression data, in: Hasan Jamil, Vasileios Magalooikonomou (Eds.), Proceedings of the 3rd IEEE Symposium on Bioinformatics and Bioengineering (BIBE'03), Bethesda, MD, USA, March 2003, pp. 321–327.
- [26] Unil Yun, Efficient mining of weighted interesting patterns with a strong weight and/or support affinity, Information Sciences 177 (17) (2007) 3477–3499.
- [27] M. Zaki, C. Hsiao, Charm: an efficient algorithm for closed association rule mining, in: Robert L. Grossman, Jiawei Han, Vipin Kumar, Heikki Mannila, Rajeev Motwani (Eds.), Proceedings of 2002 SIAM Data Mining Conference, Arlington, VA, 2002, pp. 457–473.